



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de la Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: June 12, 2026

Contents

Objetivos de Aprendizaje	4
Introducción	4
Marco Teórico	4
Topología de prueba	5
Desarrollo	6
Ejercicios de Cifrado	6
Ejercicios de Descifrado	10
Algoritmo en Python	15
Comunicación Cifrada entre Máquinas Virtuales	16
Ubuntu Desktop	16
Kali Linux	17
Repotenciar el Algoritmo en Python	18
Uso de CyberChef	21
Conclusiones	24
Referencias Bibliográficas	25

List of Figures

1	Cifrado Manual de Palabra “SOFTWARE”	6
2	Cifrado Manual de Palabra “SEGURIDAD”	7
3	Cifrado Manual de Palabra “CRIPTOGRAFIA”	8
4	Cifrado Manual de Palabra “I LOVE LINUX”	9
5	Cifrado Manual de Palabra “GOD IS LOVE”	10
6	Descifrado Manual de Palabra “KROD PXQGR”	11
7	Descifrado Manual de Palabra “FUBSWRVHJXULGDG”	12
8	Descifrado Manual de Palabra “SURJUDPDFLRQ HQ S BWKRQ” . . .	13
9	Descifrado Manual de Palabra “OLQXA LV DZHVRPH”	14

10	Descifrado Manual de Palabra “VDIH QHWZRUN”	15
11	Ejecución de Programa Python de Ejercicio 2	16
12	Escucha de Conexiones en Ubuntu Desktop	17
13	Envío del mensaje desde Kali Linux	17
14	Recibo del mensaje y conexión en Ubuntu Desktop	18
15	Ejecución del Programa Python de Ejercicio 4	21
16	Ejecución de Algoritmo “ROT13” en CyberChef	22
17	Ejecución de Algoritmo “Caesar Cipher” en CyberChef	22
18	Ejecución de Algoritmo “Base64 Encode” en CyberChef	23
19	Ejecución de Algoritmo “Base64 Decode” en CyberChef	24

List of Tables

Objetivos de Aprendizaje

Al finalizar esta práctica, usted será capaz de:

1. Comprender el proceso de encriptación y desencriptación mediante el cifrado César.
2. Entender el funcionamiento del algoritmo implementado en Python.
3. Repotenciar y mejorar el algoritmo básico del Código César.
4. Aprender a utilizar CyberChef para realizar operaciones criptográficas básicas.
5. Implementar comunicación cifrada entre máquinas virtuales utilizando Netcat (nc).

Introducción

La criptografía es una disciplina fundamental en la seguridad informática moderna. Desde los métodos clásicos hasta los sistemas criptográficos actuales, el objetivo principal consiste en proteger la información frente a accesos no autorizados.

Uno de los métodos históricos más conocidos es el Cifrado César, utilizado por Julio César para proteger mensajes militares. Aunque actualmente se considera inseguro frente a ataques modernos, sigue siendo un excelente mecanismo didáctico para comprender conceptos fundamentales de criptografía, cifrado y análisis criptográfico.

En esta práctica se implementará el algoritmo en Python, se analizará su funcionamiento y se utilizará la herramienta CyberChef para experimentar con procesos de cifrado y descifrado.

Marco Teórico

Criptología: es la ciencia que estudia los métodos de protección y análisis de información. Se divide principalmente en:

- Criptografía
- Criptoanálisis

Criptografía: consiste en desarrollar técnicas que permitan proteger la información mediante algoritmos de cifrado. Su objetivo es garantizar:

- Confidencialidad
- Integridad
- Autenticación
- No repudio

Criptoanálisis: El criptoanálisis estudia métodos para romper sistemas criptográficos o descubrir información sin conocer la clave utilizada.

Cifrado César: es un método de sustitución mono alfabética en el cual cada letra del mensaje original se desplaza un número fijo de posiciones en el alfabeto.

Fórmula del cifrado:

$$C(x) = (x + n) \bmod 26(1)$$

$$D(x) = (x - n) \bmod 26(2)$$

Donde:

- C: Cifrado
- D: Descifrado
- x: posición de la letra
- k: clave de desplazamiento
- 26: número de letras del alfabeto

CyberChef: es una herramienta web desarrollada por GCHQ utilizada para transformar, analizar, codificar, decodificar y cifrar información.

Características:

- Interfaz gráfica intuitiva
- Soporte para múltiples algoritmos
- Conversión de formatos
- Operaciones criptográficas
- Análisis de texto y datos.

Netcat (nc): es una herramienta de red utilizada para:

- Escaneo de puertos
- Transferencia de archivos
- Comunicación cliente-servidor
- Envío y recepción de mensajes TCP/UDP

En esta práctica se utilizará para enviar mensajes cifrados entre máquinas virtuales.

Topología de prueba

- Ubuntu Desktop: estación de trabajo del estudiante para desarrollo y pruebas.
- Metasploitable: Máquina objetivo de laboratorio.
- Kali Linux: máquina atacante / análisis y desarrollo

Desarrollo

Ejercicios de Cifrado

Cifrar manualmente las siguientes frases utilizando el Código César con un desplazamiento de 3 posiciones:

- SOFTWARE

```
Texto original: SOFTWARE
Desplazamiento: 3

Proceso de cifrado:
-----

Carácter 1: 'S'
Posición original: 18
 $(18 + 3) \% 26 = 21$ 
Carácter cifrado: 'V'

Carácter 2: 'O'
Posición original: 14
 $(14 + 3) \% 26 = 17$ 
Carácter cifrado: 'R'

Carácter 3: 'F'
Posición original: 5
 $(5 + 3) \% 26 = 8$ 
Carácter cifrado: 'I'

Carácter 4: 'T'
Posición original: 19
 $(19 + 3) \% 26 = 22$ 
Carácter cifrado: 'W'

Carácter 5: 'W'
Posición original: 22
 $(22 + 3) \% 26 = 25$ 
Carácter cifrado: 'Z'

Carácter 6: 'A'
Posición original: 0
 $(0 + 3) \% 26 = 3$ 
Carácter cifrado: 'D'

Carácter 7: 'R'
Posición original: 17
 $(17 + 3) \% 26 = 20$ 
Carácter cifrado: 'U'

Carácter 8: 'E'
Posición original: 4
 $(4 + 3) \% 26 = 7$ 
Carácter cifrado: 'H'

-----
Resultado final: VRIWZDUH
```

Figure 1: Cifrado Manual de Palabra “SOFTWARE”

- SEGURIDAD

```
Texto original: SEGURIDAD
Desplazamiento: 3
```

```
Proceso de cifrado:
```

```
-----
```

```
Carácter 1: 'S'
Posición original: 18
 $(18 + 3) \% 26 = 21$ 
Carácter cifrado: 'V'
```

```
Carácter 2: 'E'
Posición original: 4
 $(4 + 3) \% 26 = 7$ 
Carácter cifrado: 'H'
```

```
Carácter 3: 'G'
Posición original: 6
 $(6 + 3) \% 26 = 9$ 
Carácter cifrado: 'J'
```

```
Carácter 4: 'U'
Posición original: 20
 $(20 + 3) \% 26 = 23$ 
Carácter cifrado: 'X'
```

```
Carácter 5: 'R'
Posición original: 17
 $(17 + 3) \% 26 = 20$ 
Carácter cifrado: 'U'
```

```
Carácter 6: 'I'
Posición original: 8
 $(8 + 3) \% 26 = 11$ 
Carácter cifrado: 'L'
```

```
Carácter 7: 'D'
Posición original: 3
 $(3 + 3) \% 26 = 6$ 
Carácter cifrado: 'G'
```

```
Carácter 8: 'A'
Posición original: 0
 $(0 + 3) \% 26 = 3$ 
Carácter cifrado: 'D'
```

```
Carácter 9: 'D'
Posición original: 3
 $(3 + 3) \% 26 = 6$ 
Carácter cifrado: 'G'
```

```
-----
```

```
Resultado final: VHJXULGDG
```

Figure 2: Cifrado Manual de Palabra “SEGURIDAD”

- CRIPTOGRAFIA

```
Texto original: CRIPTOGRAFIA
Desplazamiento: 3
```

```
Proceso de cifrado:
```

```
-----
```

```
Carácter 1: 'C'
Posición original: 2
 $(2 + 3) \% 26 = 5$ 
Carácter cifrado: 'F'
```

```
Carácter 2: 'R'
Posición original: 17
 $(17 + 3) \% 26 = 20$ 
Carácter cifrado: 'U'
```

```
Carácter 3: 'I'
Posición original: 8
 $(8 + 3) \% 26 = 11$ 
Carácter cifrado: 'L'
```

```
Carácter 4: 'P'
Posición original: 15
 $(15 + 3) \% 26 = 18$ 
Carácter cifrado: 'S'
```

```
Carácter 5: 'T'
Posición original: 19
 $(19 + 3) \% 26 = 22$ 
Carácter cifrado: 'W'
```

```
Carácter 6: 'O'
Posición original: 14
 $(14 + 3) \% 26 = 17$ 
Carácter cifrado: 'R'
```

```
Carácter 7: 'G'
Posición original: 6
 $(6 + 3) \% 26 = 9$ 
Carácter cifrado: 'J'
```

```
Carácter 8: 'R'
Posición original: 17
 $(17 + 3) \% 26 = 20$ 
Carácter cifrado: 'U'
```

```
Carácter 9: 'A'
Posición original: 0
 $(0 + 3) \% 26 = 3$ 
Carácter cifrado: 'D'
```

```
Carácter 10: 'F'
Posición original: 5
 $(5 + 3) \% 26 = 8$ 
Carácter cifrado: 'I'
```

```
Carácter 11: 'I'
Posición original: 8
 $(8 + 3) \% 26 = 11$ 
Carácter cifrado: 'L'
```

Figure 3: Cifrado Manual de Palabra “CRIPTOGRAFIA”

- I LOVE LINUX

```
Carácter 1: 'I'
Posición original: 8
 $(8 + 3) \% 26 = 11$ 
Carácter cifrado: 'L'

Carácter 2: ' '
No es una letra, se conserva igual.

Carácter 3: 'L'
Posición original: 11
 $(11 + 3) \% 26 = 14$ 
Carácter cifrado: 'O'

Carácter 4: 'O'
Posición original: 14
 $(14 + 3) \% 26 = 17$ 
Carácter cifrado: 'R'

Carácter 5: 'V'
Posición original: 21
 $(21 + 3) \% 26 = 24$ 
Carácter cifrado: 'Y'

Carácter 6: 'E'
Posición original: 4
 $(4 + 3) \% 26 = 7$ 
Carácter cifrado: 'H'

Carácter 7: ' '
No es una letra, se conserva igual.

Carácter 8: 'L'
Posición original: 11
 $(11 + 3) \% 26 = 14$ 
Carácter cifrado: 'O'

Carácter 9: 'I'
Posición original: 8
 $(8 + 3) \% 26 = 11$ 
Carácter cifrado: 'L'

Carácter 10: 'N'
Posición original: 13
 $(13 + 3) \% 26 = 16$ 
Carácter cifrado: 'Q'

Carácter 11: 'U'
Posición original: 20
 $(20 + 3) \% 26 = 23$ 
Carácter cifrado: 'X'

Carácter 12: 'X'
Posición original: 23
 $(23 + 3) \% 26 = 0$ 
Carácter cifrado: 'A'

-----
Resultado final: L ORYH OLQXA
```

Figure 4: Cifrado Manual de Palabra “I LOVE LINUX”

- GOD IS LOVE

```

Texto original: GOD IS LOVE
Desplazamiento: 3

Proceso de cifrado:
-----

Carácter 1: 'G'
Posición original: 6
 $(6 + 3) \% 26 = 9$ 
Carácter cifrado: 'J'

Carácter 2: 'O'
Posición original: 14
 $(14 + 3) \% 26 = 17$ 
Carácter cifrado: 'R'

Carácter 3: 'D'
Posición original: 3
 $(3 + 3) \% 26 = 6$ 
Carácter cifrado: 'G'

Carácter 4: ' '
No es una letra, se conserva igual.

Carácter 5: 'I'
Posición original: 8
 $(8 + 3) \% 26 = 11$ 
Carácter cifrado: 'L'

Carácter 6: 'S'
Posición original: 18
 $(18 + 3) \% 26 = 21$ 
Carácter cifrado: 'V'

Carácter 7: ' '
No es una letra, se conserva igual.

Carácter 8: 'L'
Posición original: 11
 $(11 + 3) \% 26 = 14$ 
Carácter cifrado: 'O'

Carácter 9: 'O'
Posición original: 14
 $(14 + 3) \% 26 = 17$ 
Carácter cifrado: 'R'

Carácter 10: 'V'
Posición original: 21
 $(21 + 3) \% 26 = 24$ 
Carácter cifrado: 'Y'

Carácter 11: 'E'
Posición original: 4
 $(4 + 3) \% 26 = 7$ 
Carácter cifrado: 'H'

-----
Resultado final: JRG LV ORYH

```

Figure 5: Cifrado Manual de Palabra “GOD IS LOVE”

Ejercicios de Descifrado

Descifrar los siguientes mensajes aplicando manualmente el desplazamiento inverso del Código César:

- KROD PXQGR

```

Texto cifrado: KROD PXQGR
Desplazamiento: 3

Proceso de descifrado:
-----

Carácter 1: 'K'
Posición cifrada: 10
 $(10 - 3) \% 26 = 7$ 
Carácter descifrado: 'H'

Carácter 2: 'R'
Posición cifrada: 17
 $(17 - 3) \% 26 = 14$ 
Carácter descifrado: 'O'

Carácter 3: 'O'
Posición cifrada: 14
 $(14 - 3) \% 26 = 11$ 
Carácter descifrado: 'L'

Carácter 4: 'D'
Posición cifrada: 3
 $(3 - 3) \% 26 = 0$ 
Carácter descifrado: 'A'

Carácter 5: ' '
No es una letra, se conserva igual.

Carácter 6: 'P'
Posición cifrada: 15
 $(15 - 3) \% 26 = 12$ 
Carácter descifrado: 'M'

Carácter 7: 'X'
Posición cifrada: 23
 $(23 - 3) \% 26 = 20$ 
Carácter descifrado: 'U'

Carácter 8: 'Q'
Posición cifrada: 16
 $(16 - 3) \% 26 = 13$ 
Carácter descifrado: 'N'

Carácter 9: 'G'
Posición cifrada: 6
 $(6 - 3) \% 26 = 3$ 
Carácter descifrado: 'D'

Carácter 10: 'R'
Posición cifrada: 17
 $(17 - 3) \% 26 = 14$ 
Carácter descifrado: 'O'

-----
Mensaje original: HOLA MUNDO

```

Figure 6: Descifrado Manual de Palabra “KROD PXQGR”

- FUBSWRVHJXULGDG

```

Carácter 5: 'W'
Posición cifrada: 22
 $(22 - 3) \% 26 = 19$ 
Carácter descifrado: 'T'

Carácter 6: 'R'
Posición cifrada: 17
 $(17 - 3) \% 26 = 14$ 
Carácter descifrado: 'O'

Carácter 7: 'V'
Posición cifrada: 21
 $(21 - 3) \% 26 = 18$ 
Carácter descifrado: 'S'

Carácter 8: 'H'
Posición cifrada: 7
 $(7 - 3) \% 26 = 4$ 
Carácter descifrado: 'E'

Carácter 9: 'J'
Posición cifrada: 9
 $(9 - 3) \% 26 = 6$ 
Carácter descifrado: 'G'

Carácter 10: 'X'
Posición cifrada: 23
 $(23 - 3) \% 26 = 20$ 
Carácter descifrado: 'U'

Carácter 11: 'U'
Posición cifrada: 20
 $(20 - 3) \% 26 = 17$ 
Carácter descifrado: 'R'

Carácter 12: 'L'
Posición cifrada: 11
 $(11 - 3) \% 26 = 8$ 
Carácter descifrado: 'I'

Carácter 13: 'G'
Posición cifrada: 6
 $(6 - 3) \% 26 = 3$ 
Carácter descifrado: 'D'

Carácter 14: 'D'
Posición cifrada: 3
 $(3 - 3) \% 26 = 0$ 
Carácter descifrado: 'A'

Carácter 15: 'G'
Posición cifrada: 6
 $(6 - 3) \% 26 = 3$ 
Carácter descifrado: 'D'

-----
Mensaje original: CRYPTOSEGURIDAD

```

Figure 7: Descifrado Manual de Palabra “FUBSWRVHJXULGDG”

- SURJUDPDFLRQ HQ S BWKRQ


```

Carácter 12: 'Q'
Posición cifrada: 16
 $(16 - 3) \% 26 = 13$ 
Carácter descifrado: 'N'

Carácter 13: ' '
No es una letra, se conserva igual.

Carácter 14: 'H'
Posición cifrada: 7
 $(7 - 3) \% 26 = 4$ 
Carácter descifrado: 'E'

Carácter 15: 'Q'
Posición cifrada: 16
 $(16 - 3) \% 26 = 13$ 
Carácter descifrado: 'N'

Carácter 16: ' '
No es una letra, se conserva igual.

Carácter 17: 'S'
Posición cifrada: 18
 $(18 - 3) \% 26 = 15$ 
Carácter descifrado: 'P'

Carácter 18: ' '
No es una letra, se conserva igual.

Carácter 19: 'B'
Posición cifrada: 1
 $(1 - 3) \% 26 = 24$ 
Carácter descifrado: 'Y'

Carácter 20: 'W'
Posición cifrada: 22
 $(22 - 3) \% 26 = 19$ 
Carácter descifrado: 'T'

Carácter 21: 'K'
Posición cifrada: 10
 $(10 - 3) \% 26 = 7$ 
Carácter descifrado: 'H'

Carácter 22: 'R'
Posición cifrada: 17
 $(17 - 3) \% 26 = 14$ 
Carácter descifrado: 'O'

Carácter 23: 'Q'
Posición cifrada: 16
 $(16 - 3) \% 26 = 13$ 
Carácter descifrado: 'N'

-----
Mensaje original: PROGRAMACION EN P YTHON

```

Figure 8: Descifrado Manual de Palabra “SURJUDPDFLRQ HQ S BWKRQ”

- OLQXA LV DZHVRPH

```

(0 - 3) % 26 = 23
Carácter descifrado: 'X'

Carácter 6: ' '
No es una letra, se conserva igual.

Carácter 7: 'L'
Posición cifrada: 11
(11 - 3) % 26 = 8
Carácter descifrado: 'I'

Carácter 8: 'V'
Posición cifrada: 21
(21 - 3) % 26 = 18
Carácter descifrado: 'S'

Carácter 9: ' '
No es una letra, se conserva igual.

Carácter 10: 'D'
Posición cifrada: 3
(3 - 3) % 26 = 0
Carácter descifrado: 'A'

Carácter 11: 'Z'
Posición cifrada: 25
(25 - 3) % 26 = 22
Carácter descifrado: 'W'

Carácter 12: 'H'
Posición cifrada: 7
(7 - 3) % 26 = 4
Carácter descifrado: 'E'

Carácter 13: 'V'
Posición cifrada: 21
(21 - 3) % 26 = 18
Carácter descifrado: 'S'

Carácter 14: 'R'
Posición cifrada: 17
(17 - 3) % 26 = 14
Carácter descifrado: 'O'

Carácter 15: 'P'
Posición cifrada: 15
(15 - 3) % 26 = 12
Carácter descifrado: 'M'

Carácter 16: 'H'
Posición cifrada: 7
(7 - 3) % 26 = 4
Carácter descifrado: 'E'

-----
Mensaje original: LINUX IS AWESOME

```

Figure 9: Descifrado Manual de Palabra “OLQXA LV DZHVRPH”

- VDIH QHWZRUN

```

Carácter 2: 'D'
Posición cifrada: 3
 $(3 - 3) \% 26 = 0$ 
Carácter descifrado: 'A'

Carácter 3: 'I'
Posición cifrada: 8
 $(8 - 3) \% 26 = 5$ 
Carácter descifrado: 'F'

Carácter 4: 'H'
Posición cifrada: 7
 $(7 - 3) \% 26 = 4$ 
Carácter descifrado: 'E'

Carácter 5: ' '
No es una letra, se conserva igual.

Carácter 6: 'Q'
Posición cifrada: 16
 $(16 - 3) \% 26 = 13$ 
Carácter descifrado: 'N'

Carácter 7: 'H'
Posición cifrada: 7
 $(7 - 3) \% 26 = 4$ 
Carácter descifrado: 'E'

Carácter 8: 'W'
Posición cifrada: 22
 $(22 - 3) \% 26 = 19$ 
Carácter descifrado: 'T'

Carácter 9: 'Z'
Posición cifrada: 25
 $(25 - 3) \% 26 = 22$ 
Carácter descifrado: 'W'

Carácter 10: 'R'
Posición cifrada: 17
 $(17 - 3) \% 26 = 14$ 
Carácter descifrado: 'O'

Carácter 11: 'U'
Posición cifrada: 20
 $(20 - 3) \% 26 = 17$ 
Carácter descifrado: 'R'

Carácter 12: 'N'
Posición cifrada: 13
 $(13 - 3) \% 26 = 10$ 
Carácter descifrado: 'K'

-----
Mensaje original: SAFE NETWORK

```

Figure 10: Descifrado Manual de Palabra “VDIH QHWZRUN”

Algoritmo en Python

Código Base:

```

def cifrado_cesar(texto, desplazamiento):
    resultado = ""

    for caracter in texto:

```

```

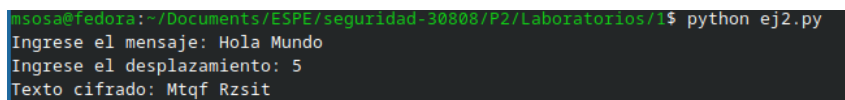
    if caracter.isalpha():
        base = ord('A') if caracter.isupper() else ord('a')
        nuevo_caracter = chr(
            (ord(caracter) - base + desplazamiento) % 26 + base
        )

        resultado += nuevo_caracter
    else:
        resultado += caracter
return resultado

mensaje = input("Ingrese el mensaje: ")
clave = int(input("Ingrese el desplazamiento: "))

texto_cifrado = cifrado_cesar(mensaje, clave)
print("Texto cifrado:", texto_cifrado)

```



```

msosa@fedora:~/Documents/ESPE/seguridad-30808/P2/Laboratorios/1$ python ej2.py
Ingrese el mensaje: Hola Mundo
Ingrese el desplazamiento: 5
Texto cifrado: Mtqf Rzsit

```

Figure 11: Ejecución de Programa Python de Ejercicio 2

Analizar el código y responder:

1. ¿Qué función cumple ord()?

Convierte un carácter en su código numérico Unicode (para letras inglesas coincide con ASCII).

2. ¿Qué función cumple chr()?

Realiza la operación inversa de ord(): convierte un número Unicode en un carácter.

3. ¿Qué representa % 26?

El operador % es el módulo (resto de una división).

4. ¿Cómo funciona el desplazamiento?

El desplazamiento indica cuántas posiciones debe moverse cada letra dentro del alfabeto.

5. ¿Qué ocurre con símbolos y espacios?

Los símbolos, espacios, números y signos de puntuación no se cifran.

Comunicación Cifrada entre Máquinas Virtuales

Ubuntu Desktop

Escuchar conexiones:

```
nc -lvnp 4444
```

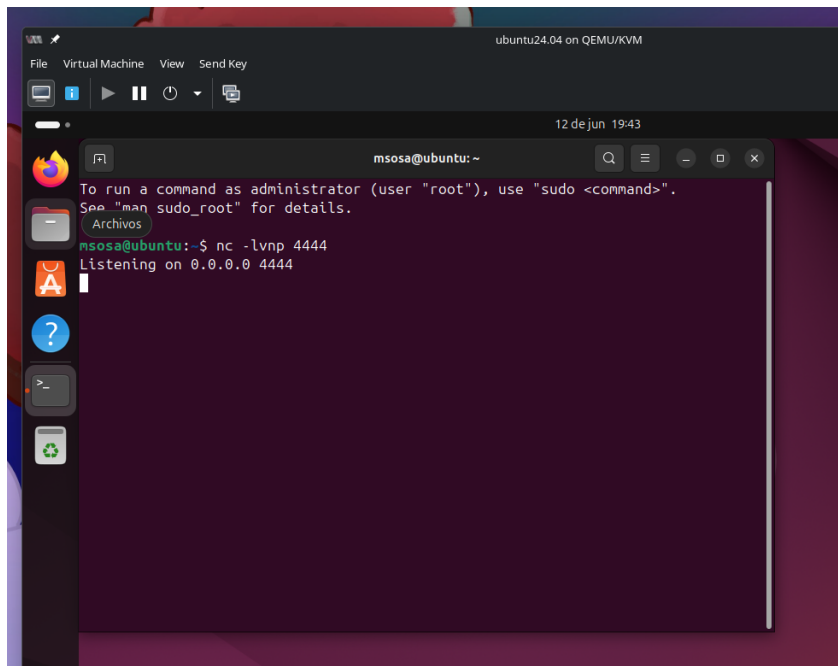


Figure 12: Escucha de Conexiones en Ubuntu Desktop

Kali Linux

Enviar mensaje:

echo "PHQVDMH HQFULSWDGR" | nc <IP_UBUNTU> 4444

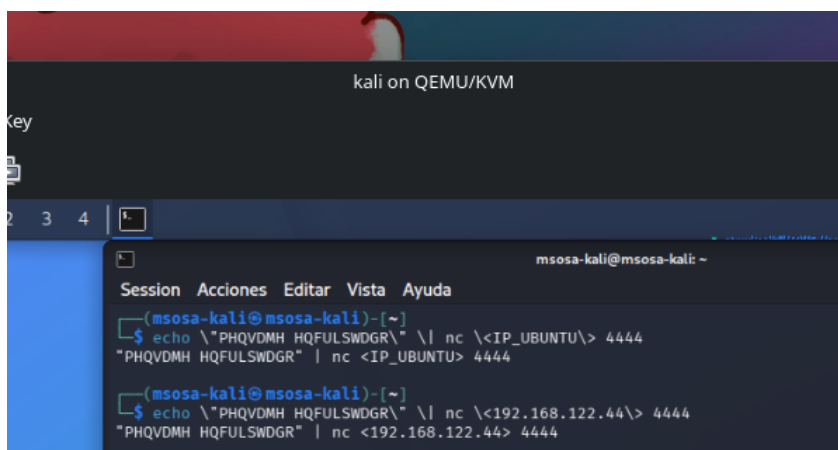


Figure 13: Envío del mensaje desde Kali Linux

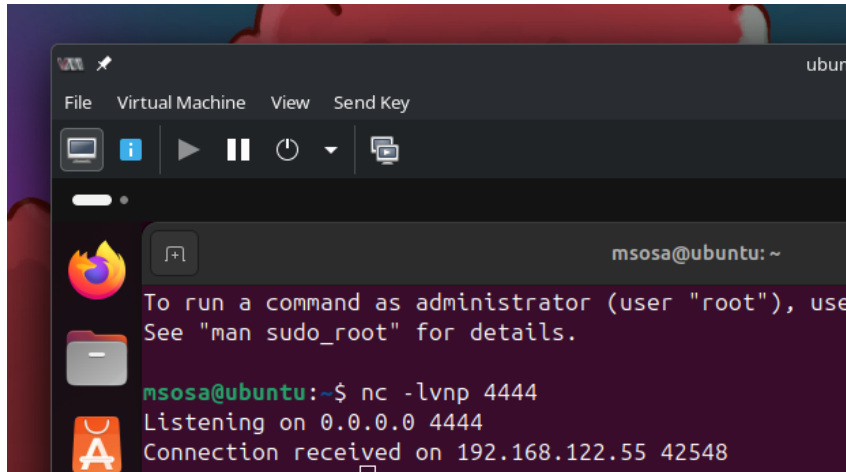


Figure 14: Recibo del mensaje y conexión en Ubuntu Desktop

Repotenciar el Algoritmo en Python

Modificar el programa para:

- Permitir cifrado y descifrado.
- Incorporar menú interactivo.
- Validar entradas incorrectas.
- Soportar mayúsculas y minúsculas.
- Manejar caracteres especiales.
- Ejecutar las pruebas de evaluación y validación correspondientes.

El nuevo programa es el siguiente:

```
def cifrado_cesar(texto, desplazamiento):
    resultado = ""

    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')

            nuevo_caracter = chr(
                (ord(caracter) - base + desplazamiento) % 26 + base
            )

            resultado += nuevo_caracter
        else:
            # Mantener espacios, números y símbolos
            resultado += caracter

    return resultado
```

```

def descifrado_cesar(texto, desplazamiento):
    return cifrado_cesar(texto, -desplazamiento)

def obtener_desplazamiento():
    while True:
        try:
            valor = int(input("Ingrese el desplazamiento (entero): "))
            return valor
        except ValueError:
            print("Error: debe ingresar un número entero.")

def pruebas():
    print("\n=== PRUEBAS AUTOMÁTICAS ===")

    mensaje = "Hola Mundo"
    clave = 3

    cifrado = cifrado_cesar(mensaje, clave)
    descifrado = descifrado_cesar(cifrado, clave)

    print(f"Mensaje original : {mensaje}")
    print(f"Cifrado           : {cifrado}")
    print(f"Descifrado         : {descifrado}")

    assert descifrado == mensaje

    # Prueba de mayúsculas y minúsculas
    assert cifrado_cesar("ABC xyz", 2) == "CDE zab"

    # Prueba de caracteres especiales
    assert cifrado_cesar("Hola, Mundo! 123", 5) == "Mtqf, Rzsit! 123"

    print("Todas las pruebas fueron superadas.\n")

def menu():
    while True:
        print("\n===== CIFRADO CÉSAR =====")
        print("1. Cifrar mensaje")
        print("2. Descifrar mensaje")
        print("3. Ejecutar pruebas")
        print("4. Salir")

        opcion = input("Seleccione una opción: ")

        if opcion == "1":

```

```

    mensaje = input("Ingrese el mensaje: ")
    clave = obtener_desplazamiento()

    resultado = cifrado_cesar(mensaje, clave)
    print("Mensaje cifrado:", resultado)

elif opcion == "2":
    mensaje = input("Ingrese el mensaje cifrado: ")
    clave = obtener_desplazamiento()

    resultado = descifrado_cesar(mensaje, clave)
    print("Mensaje descifrado:", resultado)

elif opcion == "3":
    pruebas()

elif opcion == "4":
    print("Programa finalizado.")
    break

else:
    print("Opción inválida. Intente nuevamente.")

if __name__ == "__main__":
    menu()

```



```

===== CIFRADO CÉSAR =====
1. Cifrar mensaje
2. Descifrar mensaje
3. Ejecutar pruebas
4. Salir
Seleccione una opción: 1
Ingrese el mensaje: HolaMundo
Ingrese el desplazamiento (entero): 5
Mensaje cifrado: MtqfRzsit

===== CIFRADO CÉSAR =====
1. Cifrar mensaje
2. Descifrar mensaje
3. Ejecutar pruebas
4. Salir
Seleccione una opción: 2
Ingrese el mensaje cifrado: MtqfRzsit
Ingrese el desplazamiento (entero): 5
Mensaje descifrado: HolaMundo

===== CIFRADO CÉSAR =====
1. Cifrar mensaje
2. Descifrar mensaje
3. Ejecutar pruebas
4. Salir
Seleccione una opción: 3

=== PRUEBAS AUTOMÁTICAS ===
Mensaje original : Hola Mundo
Cifrado          : Krod Pxqgr
Descifrado       : Hola Mundo
Todas las pruebas fueron superadas.

===== CIFRADO CÉSAR =====
1. Cifrar mensaje
2. Descifrar mensaje
3. Ejecutar pruebas
4. Salir
Seleccione una opción: 4
Programa finalizado.

```

Figure 15: Ejecución del Programa Python de Ejercicio 4

Uso de CyberChef

Ingresa a: <https://gchq.github.io/CyberChef/>

Realizar las siguientes operaciones:

1. ROT13

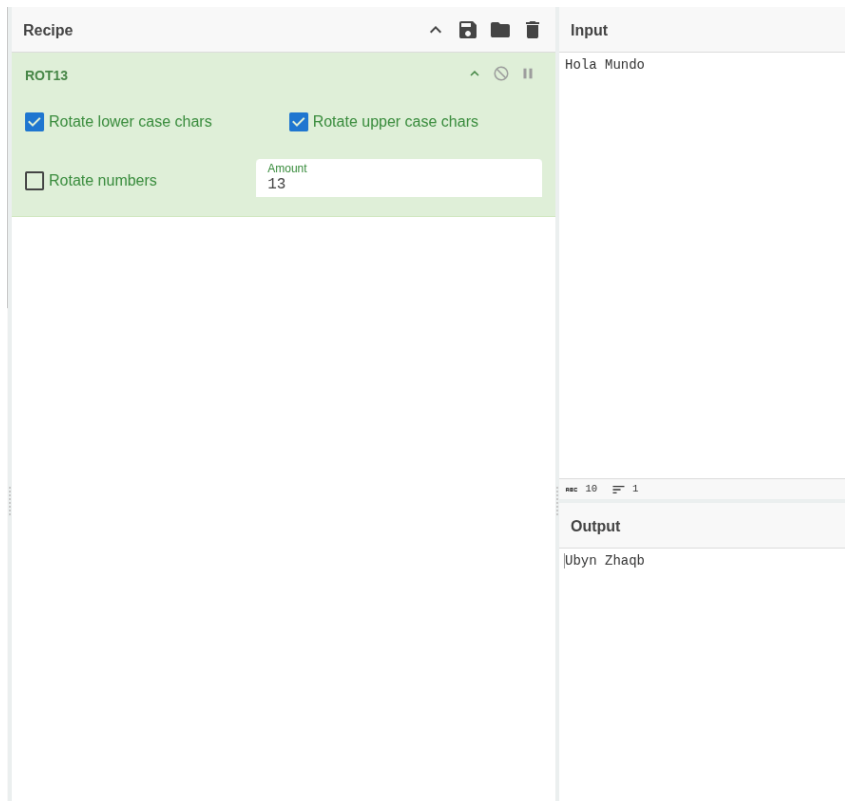


Figure 16: Ejecución de Algoritmo “ROT13” en CyberChef

2. Caesar Cipher

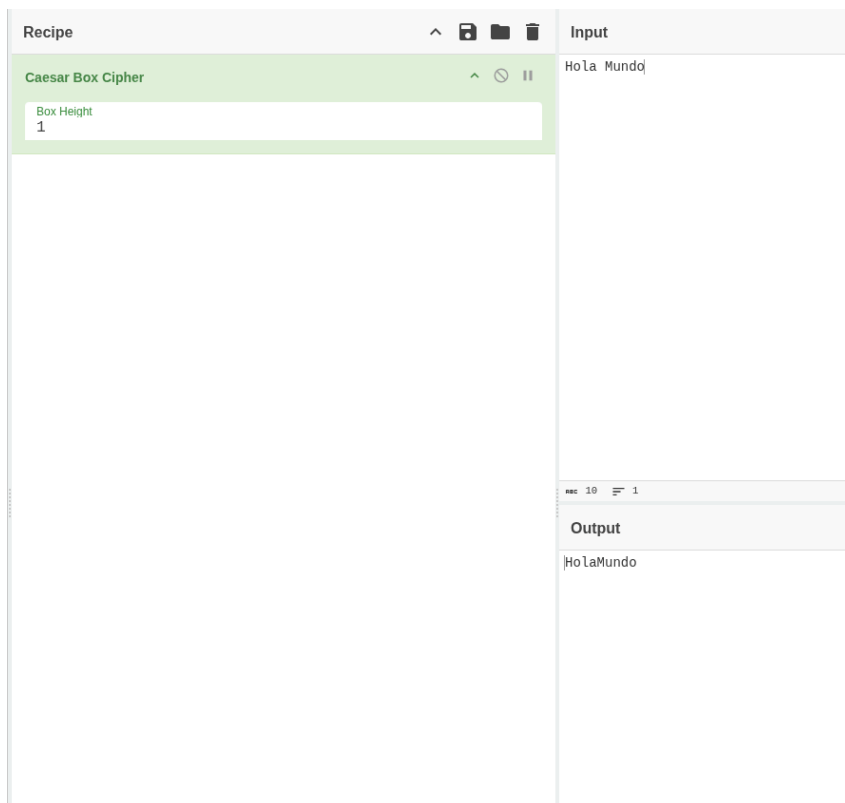


Figure 17: Ejecución de Algoritmo “Caesar Cipher” en CyberChef

3. Base64 Encode

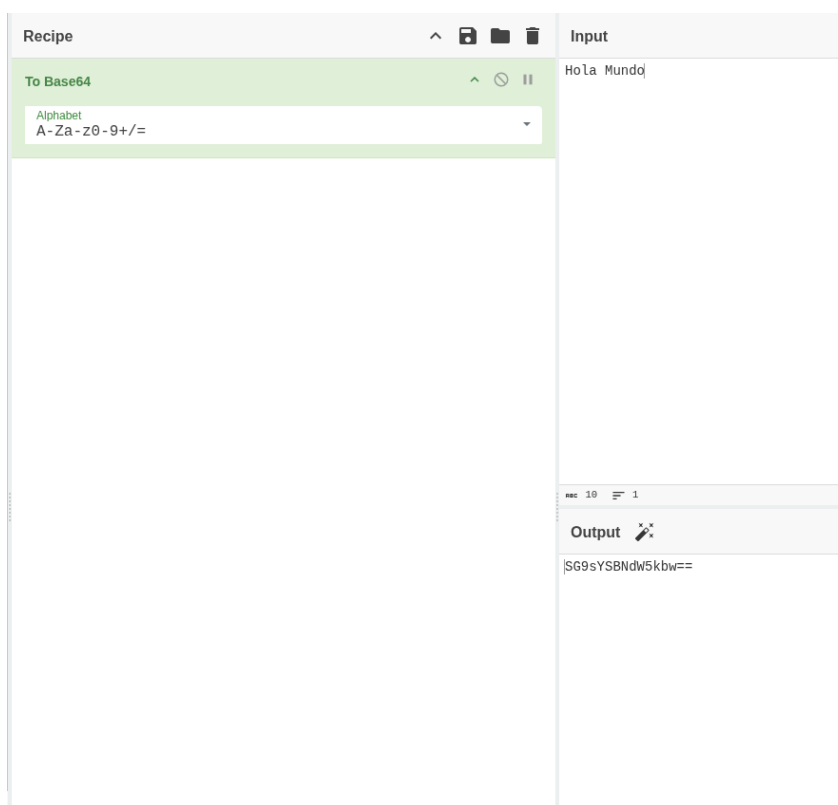


Figure 18: Ejecución de Algoritmo “Base64 Encode” en CyberChef

4. Base64 Decode

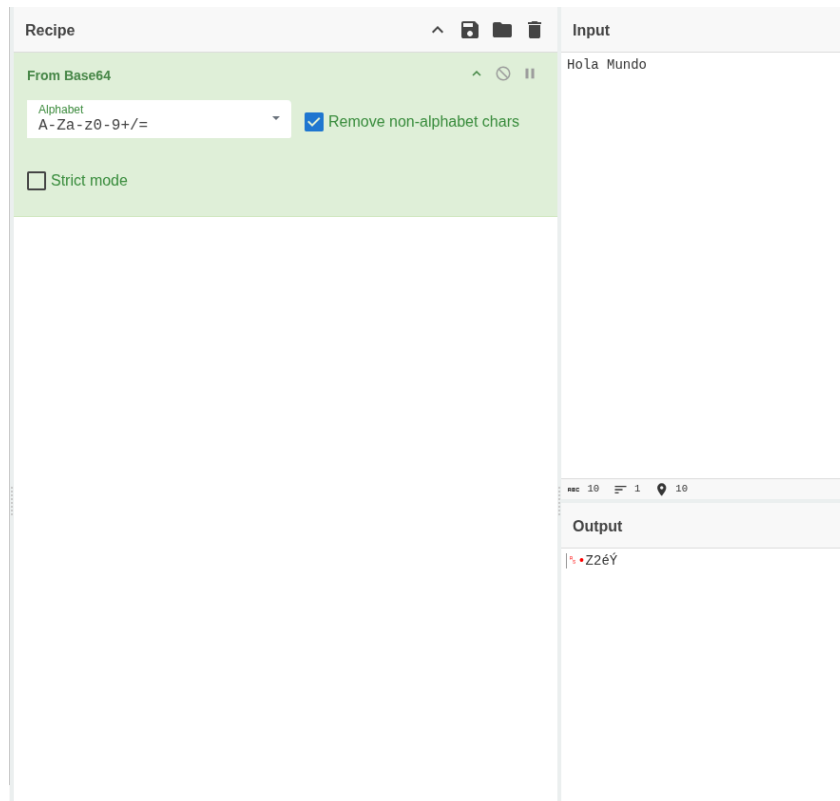


Figure 19: Ejecución de Algoritmo “Base64 Decode” en CyberChef

Investigar:

1. ¿Qué otras herramientas ofrecen CyberChef?

CyberChef ofrece herramientas de cifrado, descifrado, codificación, decodificación, cálculo de hashes y conversión de formatos.

2. ¿Qué ventajas presenta?

CyberChef es fácil de usar, gratuito y permite realizar múltiples operaciones sin programar.

3. ¿Qué limitaciones tiene?

CyberChef puede ser lento con archivos grandes y no reemplaza herramientas especializadas.

4. ¿Puede utilizarse en análisis forense?

Sí, CyberChef puede utilizarse para analizar, decodificar y transformar evidencias digitales.

Conclusiones

- El cifrado César permite comprender fundamentos esenciales de la criptografía clásica.
- Python facilita la implementación y experimentación de algoritmos criptográficos.
- Las máquinas virtuales permiten realizar pruebas seguras en entornos controlados.

- Netcat facilita comprender procesos básicos de comunicación en red.
- CyberChef constituye una herramienta útil para análisis y transformación criptográfica.
- Los algoritmos clásicos presentan vulnerabilidades importantes frente a ataques modernos.

Referencias Bibliográficas

- Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice*. Pearson.
- Schneier, B. (2015). *Applied Cryptography*. Wiley.
- Menezes, A., van Oorschot, P., & Vanstone, S. (1996). *Handbook of Applied Cryptography*. CRC Press.
- Katz, J., & Lindell, Y. (2020). *Introduction to Modern Cryptography*. CRC Press.
- Singh, S. (2000). *The Code Book*. Anchor Books.
- Python Software Foundation. *Python Documentation*



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de la Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: June 16, 2026

Contents

Objetivos de Aprendizaje	3
Marco Teórico	3
Topología de prueba	3
Desarrollo	4
Generación y Comparación de Hashes	4
Hashing en Python	6
Vulnerabilidad y Protección con Hash	6
Investigación	7
Conclusiones	7
Referencias Bibliográficas	8

List of Figures

1	Guardado de Mensaje “Seguridad en redes 2025” en Documento	4
2	Ejecución de Algoritmo de Hash “md5sum”	4
3	Ejecución de Algoritmo de Hash “sha1sum”	5
4	Ejecución de Algoritmo de Hash “sha256sum”	5
5	Envío de Documento desde Kali a Ubuntu	5
6	Encriptado de Documento	6
7	Ejecución de Script Python de Ejercicio 2	6
8	Ejecución de Script Python de Ejercicio 3	7

List of Tables

Objetivos de Aprendizaje

Al finalizar esta práctica, usted será capaz de:

1. Comprender el funcionamiento de las funciones hash criptográficas.
2. Utilizar comandos y herramientas en Linux para calcular hashes.
3. Verificar la integridad de archivos mediante valores hash.
4. Aplicar el hashing en scripts de Python para mejorar la seguridad del código.
5. Identificar prácticas inseguras que pueden evitarse usando funciones hash.
6. Investigar y aplicar conceptos de autenticidad, integridad y verificación mediante hashes.

Marco Teórico

Las funciones hash criptográficas son funciones matemáticas que transforman datos de tamaño arbitrario en una cadena de longitud fija. Estas funciones tienen las siguientes propiedades:

- Unidireccionalidad: no se puede obtener el dato original a partir del hash.
- Determinismo: la misma entrada siempre produce la misma salida.
- Resistencia a colisiones: es computacionalmente difícil encontrar dos entradas distintas con el mismo hash.
- Avalanche: un pequeño cambio en la entrada produce un hash totalmente distinto.

En la práctica, los algoritmos hash son ampliamente usados para:

- Verificar la integridad de archivos.
- Guardar contraseñas de forma segura.
- Firmas digitales y certificados.
- Evitar la ejecución de código no autorizado.

Algunos algoritmos comunes incluyen MD5, SHA-1 y SHA-256, entre otros. Aunque algunos (como MD5 o SHA-1) ya no se consideran seguros para aplicaciones críticas, siguen siendo útiles para fines educativos o de verificación básica.

Topología de prueba

Se utilizarán las siguientes máquinas virtuales:

- Ubuntu Desktop: estación de trabajo del estudiante.
- Ubuntu Server: servidor de archivos.
- Kali Linux: entorno para validación de vulnerabilidades y pruebas de seguridad.

Desarrollo

Generación y Comparación de Hashes

En Ubuntu Desktop:

1. Crear un archivo de texto:

```
echo "Seguridad en redes 2025" > documento.txt
```

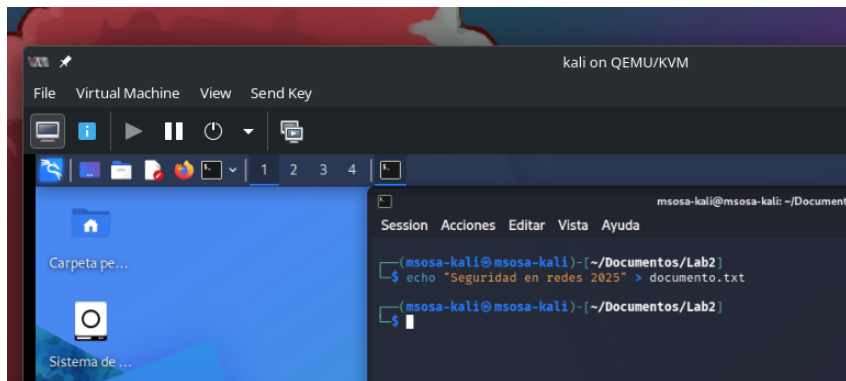


Figure 1: Guardado de Mensaje “Seguridad en redes 2025” en Documento

2. Generar el hash del archivo con diferentes algoritmos:

```
md5sum documento.txt
```

```
sha1sum documento.txt
```

```
sha256sum documento.txt
```

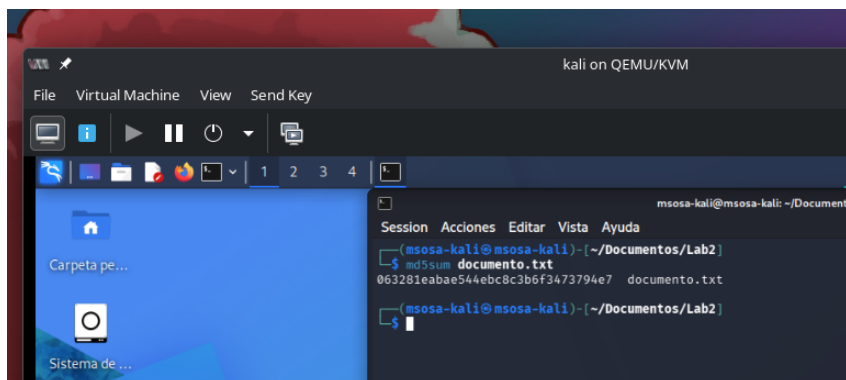


Figure 2: Ejecución de Algoritmo de Hash “md5sum”

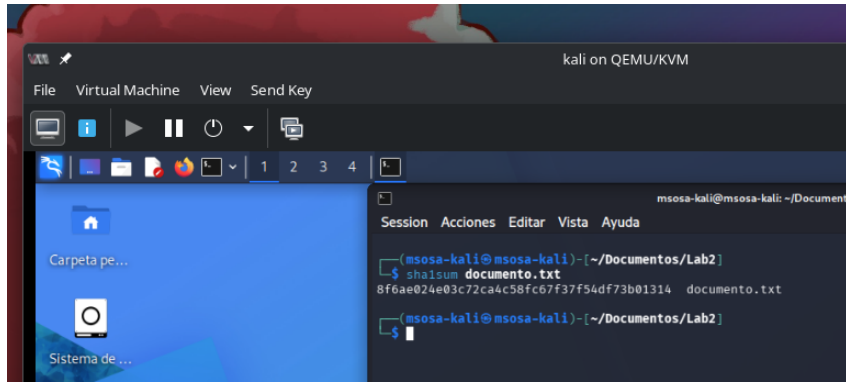


Figure 3: Ejecución de Algoritmo de Hash “sha1sum”

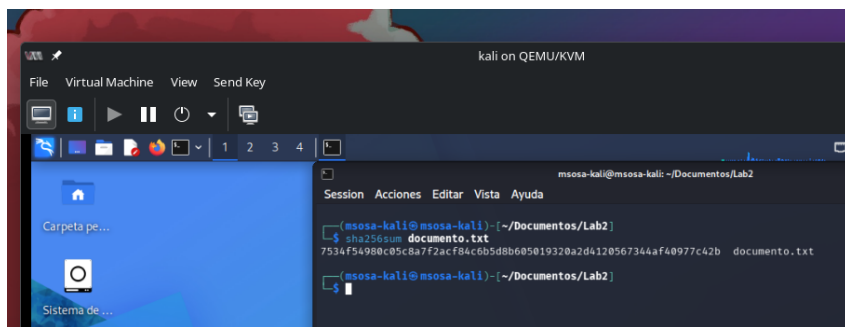


Figure 4: Ejecución de Algoritmo de Hash “sha256sum”

3. Modificar el contenido del archivo (por ejemplo, cambiar una letra), volver a generar los hashes y observar cómo cambian.
4. Transferir el archivo al servidor Ubuntu Server y verificar que no fue alterado:

```
scp documento.txt usuario@IP_SERVER:/home/usuario/ssh usuario@IP_SERVER
```

```
sha256sum documento.txt
```

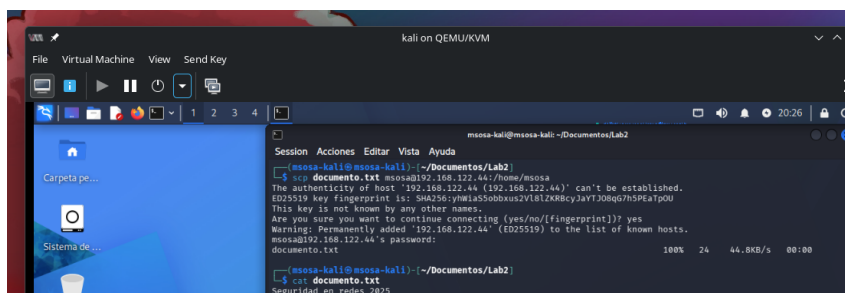


Figure 5: Envío de Documento desde Kali a Ubuntu

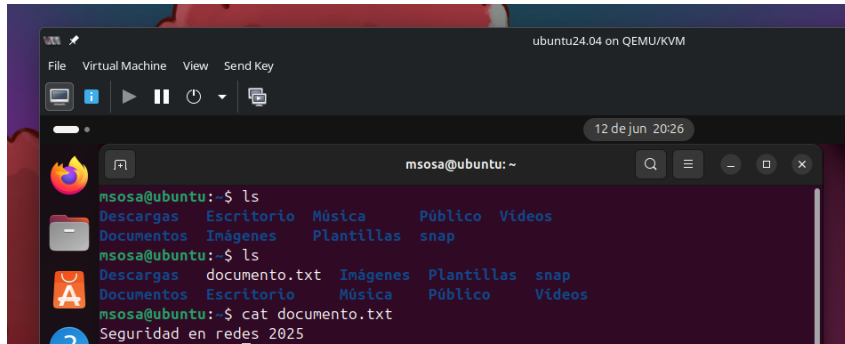


Figure 6: Encriptado de Documento

Hashing en Python

1. Crear un script en Ubuntu Desktop llamado hash_script.py:

```
import hashlib

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

passwd = input("Introduce una contraseña: ")
print("Hash SHA-256:", hash_password(passwd))
```

2. Ejecutar el script y probar con diferentes contraseñas para ver cómo cambia el hash.

```
msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/2$ python ej2.py
Introduce una contraseña: Mateo
Hash SHA-256: 8272dce0d7192d4599a7ee498066f8e7c36dcffe1a7c70f15854fd228fdfa25
msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/2$ python ej2.py
Introduce una contraseña: Hola
Hash SHA-256: e633f4fc79badea1dc5db970cf397c8248bac47cc3acf9915ba60b5d76b0e88f
msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/2$ python ej2.py
Introduce una contraseña: Test
Hash SHA-256: 532eaabd9574880dbf76b9b8cc00832c20a6ec113d682299550d7a6e0f345e25
```

Figure 7: Ejecución de Script Python de Ejercicio 2

Vulnerabilidad y Protección con Hash

Código vulnerable en Python que compara contraseñas en texto plano:

```
passwords = ["admin123", "qwerty"]
user_input = input("Contraseña: ")
if user_input in passwords:
    print("Acceso concedido")
else:
    print("Acceso denegado")
```

```
msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/2$ python ej3.py
Contraseña: Hola
Acceso denegado
msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/2$ python ej3.py
Contraseña: admin123
Acceso concedido
msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/2$
```

Figure 8: Ejecución de Script Python de Ejercicio 3

Problema: el almacenamiento y la comparación de contraseñas en texto plano representa una grave vulnerabilidad de seguridad.

Investigación

Los estudiantes deberán investigar y aplicar una solución para almacenar y comparar contraseñas de forma segura usando funciones hash. Deberán responder:

1. ¿Qué riesgos presenta almacenar contraseñas en texto plano?

Almacenar contraseñas en texto plano permite que cualquier persona con acceso a la base de datos pueda verlas y utilizarlas directamente.

2. ¿Qué ventajas ofrece el uso de hashing?

El hashing protege las contraseñas al almacenarlas como valores irreversibles, evitando que se conozca su contenido original.

3. ¿Qué algoritmos de hash son recomendables para el almacenamiento de contraseñas?

Los algoritmos más recomendados son bcrypt, Argon2 y PBKDF2, ya que están diseñados específicamente para proteger contraseñas.

4. ¿Cómo implementar una comparación segura de contraseñas en Python?

La comparación segura se implementa almacenando el hash de la contraseña y verificando la contraseña ingresada contra dicho hash mediante bibliotecas como **bcrypt** o **argon2**.

Conclusiones

1. Las funciones hash permiten asegurar la integridad y autenticidad de datos en entornos digitales.
2. Son herramientas esenciales en la seguridad informática, tanto en redes como en desarrollo de software.
3. El uso de hashes puede prevenir vulnerabilidades comunes como el almacenamiento inseguro de contraseñas.
4. Entornos como Linux y lenguajes como Python ofrecen múltiples utilidades para aplicar criptografía hash de forma eficiente y didáctica.

Referencias Bibliográficas

1. Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice*. Pearson.
2. Paar, C., & Pelzl, J. (2010). *Understanding Cryptography: A Textbook for Students and Practitioners*. Springer.
3. The Open Web Application Security Project (OWASP). (2024). *Password Storage Cheat Sheet*. <https://cheatsheetseries.owasp.org>
4. Python Software Foundation. (2024). *Python 3 Standard Library: hashlib*. <https://docs.python.org/3/library/hashlib.html>
5. Linux Man Pages. md5sum, sha1sum, sha256sum



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de la Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: June 12, 2026

Contents

Objetivo	4
Topología de Prueba	4
Objetivos de Aprendizaje	4
Marco Teórico	4
AES	4
Criptología	4
Criptografía	5
Criptoanálisis	5
Concepto de Criptografía Simétrica	5
Topología de prueba	5
Desarrollo	6
Preparación del entorno	6
Crear un archivo de texto para cifrar	6
Cifrado con AES	6
Verificación del cifrado	7
Descifrado del archivo	8
Reflexión y análisis	8
Actividades complementarias	8
Análisis y ejecución del algoritmo básico AES-256 en Python	8
Actividad de modificación y mejoramiento del algoritmo propuesto	10
Pregunta	10
Investigación	11

List of Figures

1	Verificación de Versión de <code>openssl</code>	6
---	---	---

2	Creación de Archivo de Texto a Cifrar	6
3	Encriptado de Archivo con AES	7
4	Verificación de Archivo Cifrado	7
5	Contenido de Mensaje Cifrado	7
6	Desencriptado de Mensaje Cifrado con Contraseña	8
7	Contenido de Mensaje Descifrado	8
8	Instalación de Librería cryptography	9
9	Ejecución de Script Python del Ejercicio 6	9
10	Ejecución de Script Python de Ejercicio 7	10
11	Creación del Archivo del Ejercicio 8	11
12	Encriptado del Archivo mediante Diversos Algoritmos	12
13	Revisión de Archivos Encriptados mediante hexdump	12

List of Tables

Objetivo

Aplicar el cifrado simétrico utilizando el algoritmo **AES (Advanced Encryption Standard)** en un entorno virtual controlado con Kali Linux, comprendiendo su funcionamiento y seguridad.

Topología de Prueba

Entorno Virtual de Red: Kali Linux y OpenSSL

Objetivos de Aprendizaje

1. Comprender el funcionamiento del algoritmo de cifrado simétrico AES.
2. Analizar e implementar una versión básica de AES en Python.
3. Probar el algoritmo con claves de 128, 192 y 256 bits.
4. Aplicar AES en un contexto de red para proteger la información.
5. Identificar posibles mejoras al código de cifrado implementado en Python.
6. Simular ataques para evaluar la efectividad de las técnicas de cifrado.

Marco Teórico

AES

El AES (Advanced Encryption Standard) es un algoritmo de cifrado simétrico adoptado como estándar por el NIST en 2001. Sustituyó al DES debido a sus debilidades frente a ataques por fuerza bruta. AES está basado en el algoritmo Rijndael, propuesto por Joan Daemen y Vincent Rijmen, y emplea claves de 128, 192 o 256 bits con bloques fijos de 128 bits. Dependiendo del tamaño de la clave, realiza 10, 12 o 14 rondas de operaciones criptográficas (Daemen & Rijmen, 2002; Stallings, 2017).

Las operaciones internas incluyen sustituciones, permutaciones y combinaciones lineales a nivel de byte: SubBytes, ShiftRows, MixColumns y AddRoundKey. En redes informáticas, AES se utiliza para cifrar comunicaciones seguras entre hosts o dispositivos, siendo clave en protocolos como TLS, IPsec y VPNs.

Criptología

La criptología es la ciencia que estudia los métodos y técnicas utilizadas para proteger la información mediante el uso de códigos y cifrados, con el fin de garantizar la triada CID y el No repudio de los datos. Abarca dos disciplinas principales: la criptografía (cifrado de la información) y el criptoanálisis (estudio de la decodificación de mensajes cifrados sin conocer la clave).

Criptografía

La criptografía es una rama de la criptología que se enfoca en el diseño y estudio de algoritmos matemáticos y protocolos que permiten transformar la información en un formato seguro y protegido contra accesos no autorizados.

Criptoanálisis

El criptoanálisis es la parte de la criptología que se dedica al estudio de sistemas criptográficos con el fin de encontrar debilidades en dichos sistemas y romper su seguridad sin el conocimiento de información secreta.

Concepto de Criptografía Simétrica

La criptografía simétrica es un tipo de criptografía en la que se utiliza la misma clave para cifrar y descifrar la información. Esto significa que ambas partes (el remitente y el receptor) deben compartir una clave secreta previamente acordada. El principal desafío de la criptografía simétrica es la gestión de claves, ya que la clave debe ser transmitida de forma segura sin ser interceptada.

OpenSSL es una herramienta de software de código abierto ampliamente utilizada en sistemas Linux (y otros sistemas operativos) para la implementación de funciones criptográficas. Ofrece una biblioteca criptográfica robusta que proporciona un conjunto de algoritmos de cifrado, funciones de seguridad y utilidades para la gestión de certificados **SSL/TLS**, la generación de claves y la creación de firmas digitales. Entre sus principales usos se tiene:

- OpenSSL permite realizar cifrado y descifrado de datos utilizando una variedad de algoritmos criptográficos, como **AES**, **DES**, **RSA**, **Blowfish**, entre otros.
- OpenSSL permite generar claves públicas y privadas para criptografía simétrica y asimétrica.
- OpenSSL permite generar certificados digitales utilizados en protocolos como **SSL/TLS** para asegurar la comunicación en redes.

Topología de prueba

- **VM Cliente (Ubuntu Desktop):** Ejecutará un programa Python para cifrar y enviar un mensaje usando AES.
- **VM Servidor (Ubuntu Server):** Recibirá y descifrá el mensaje enviado por el cliente, también usando Python.
- **VM Atacante (Kali Linux):** Capturará el tráfico de red (usando Wireshark o tcpdump) e intentará analizar su contenido.
- **Python 3 instalado en cliente y servidor.**
- **Biblioteca PyCryptodome:**

Desarrollo

Preparación del entorno

1. Inicie su máquina virtual Kali Linux y abra una terminal.
2. Verifique que OpenSSL esté instalado en tu máquina. Para esto, escriba el siguiente comando en la terminal:

```
openssl version
```

Si OpenSSL no está instalado, instálalo ejecutando:

```
sudo apt-get update
```

```
sudo apt-get install openssl
```

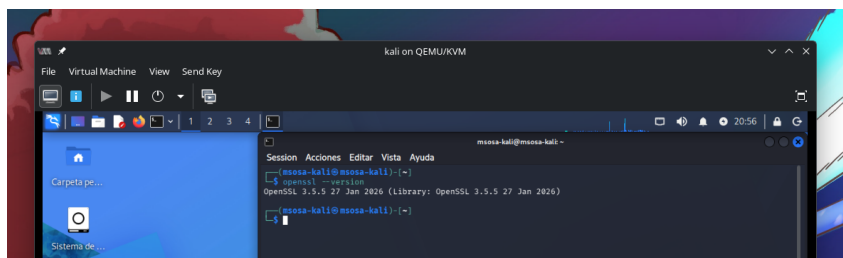


Figure 1: Verificación de Versión de openssl

Crear un archivo de texto para cifrar

1. Cree un archivo de texto con el siguiente contenido:

```
echo "Este es un mensaje secreto que vamos a cifrar usando AES" > mensaje.txt
```

2. Verifique que el archivo se haya creado correctamente:

```
cat mensaje.txt
```

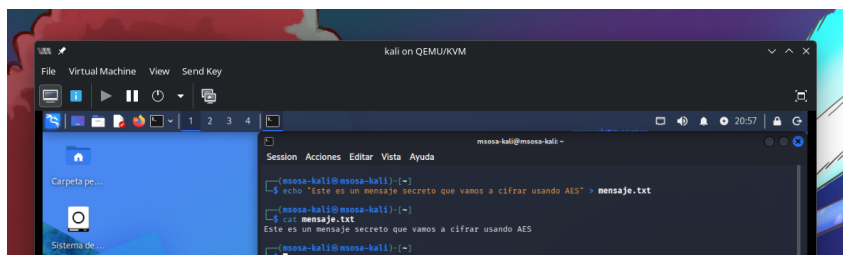


Figure 2: Creación de Archivo de Texto a Cifrar

Cifrado con AES

1. Para cifrar el archivo mensaje.txt usando **AES-256-CBC** (AES con un tamaño de clave de 256 bits y el modo de operación CBC), utilice el siguiente comando:

```
openssl enc -aes-256-cbc -salt -in mensaje.txt -out mensaje_cifrado.txt
```

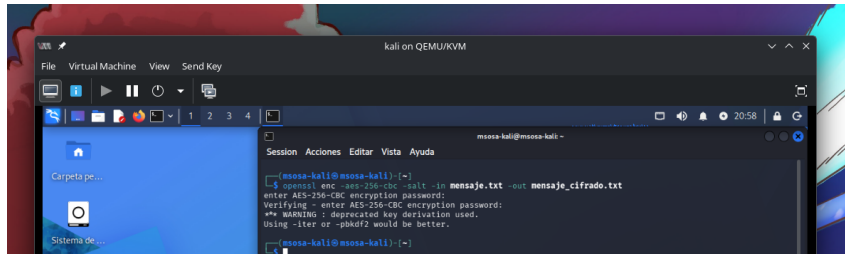


Figure 3: Encriptado de Archivo con AES

El parámetro **-salt**: añade un valor aleatorio (sal) para mejorar la seguridad del cifrado. Cuando se ejecute el comando, se pedirá que ingreses una **contraseña** (que será la clave secreta para el cifrado).

3. Después de ingresar la contraseña, el archivo se cifrará y se guardará en mensaje_cifrado.txt.
4. Verifique que el archivo cifrado haya sido creado:

```
ls -l mensaje_cifrado.txt
```

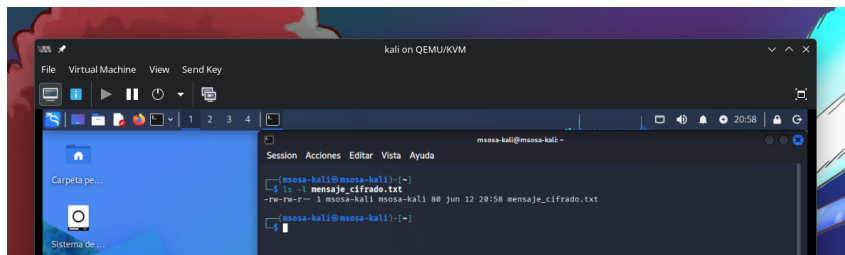


Figure 4: Verificación de Archivo Cifrado

Verificación del cifrado

1. Para verificar que el archivo se cifró correctamente, intenta abrir el archivo cifrado con un editor de texto. Verás que el contenido no es legible:

```
cat mensaje_cifrado.txt
```

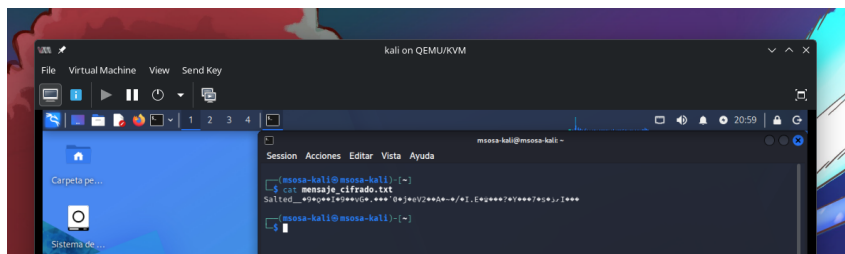


Figure 5: Contenido de Mensaje Cifrado

El archivo debe mostrar datos aparentemente aleatorios, como una secuencia de caracteres binarios o texto no legible, lo que confirma que el mensaje está cifrado.

Descifrado del archivo

1. Para descifrar el archivo, use el siguiente comando:

```
openssl enc -d -aes-256-cbc -in mensaje_cifrado.txt -out mensaje_descifrado.txt
```

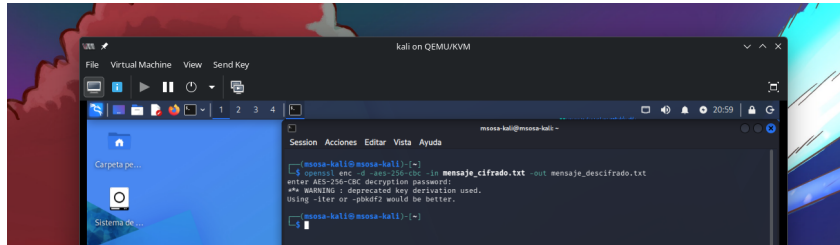


Figure 6: Desencriptado de Mensaje Cifrado con Contraseña

Se te pedirá que ingreses la misma contraseña utilizada durante el cifrado.

2. Una vez completado el proceso, Verifique que el archivo haya sido descifrado correctamente:

```
cat mensaje_descifrado.txt
```

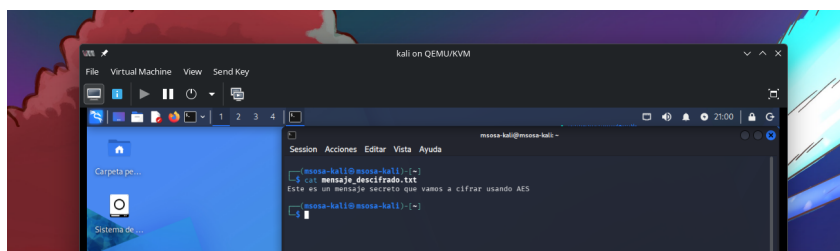


Figure 7: Contenido de Mensaje Descifrado

Reflexión y análisis

1. **Seguridad:** La criptografía simétrica es eficiente en términos de velocidad, pero su principal vulnerabilidad es la **gestión de claves**. Ambas partes que cifran y descifran deben compartir la misma clave, lo que puede generar problemas de seguridad si la clave es interceptada.

Actividades complementarias

Análisis y ejecución del algoritmo básico AES-256 en Python

Objetivo: Comprender el funcionamiento básico del cifrado simétrico AES-256 en modo GCM, mediante la ejecución de un programa en Python que permite cifrar y descifrar información.

Instalar la librería

```
pip install cryptography
```

```
(venv) msosa@fedora:~/Documents/ESPE/seguridad-30808/P2/Laboratorios/3$ pip install cryptography
Collecting cryptography
  Downloading cryptography-49.0.0-cp311-abi3-manylinux_2_34_x86_64.whl.metadata (4.3 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading cryptography-49.0.0-cp311-abi3-manylinux_2_34_x86_64.whl (4.7 MB)
3.4/4.7 MB 1.1 MB/s eta 0:00:02
```

Figure 8: Instalación de Librería cryptography

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

# Generación de clave AES-256 (32 bytes = 256 bits)
key = AESGCM.generate_key(bit_length=256)
aesgcm = AESGCM(key)

# Nonce (vector de inicialización)
nonce = os.urandom(12)

# Mensaje original
mensaje = "Seguridad informática con AES-256"
datos = mensaje.encode("utf-8")

# CIFRADO
ciphertext = aesgcm.encrypt(nonce, datos, None)

print("MENSAJE ORIGINAL:", mensaje)
print("CLAVE:", key.hex())
print("NONCE:", nonce.hex())
print("CIFRADO:", ciphertext.hex())

# DESCIFRADO
texto_descifrado = aesgcm.decrypt(nonce, ciphertext, None)
print("DESCIFRADO:", texto_descifrado.decode("utf-8"))
```

```
(venv) msosa@fedora:~/Documents/ESPE/seguridad-30808/P2/Laboratorios/3$ python ej6.py
MENSAJE ORIGINAL: Seguridad informática con AES-256
CLAVE: bbf52f89725b23f3b1e4efab0b1b067cfa68ca5d173efb4bbccd9d58b52b57bf
NONCE: 6f70895a797b44f726ce65e9
CIFRADO: 08e70a9565e1c7acd43be8a625603ef235213d7a83849a9a217edc51d00955b1f16dd44ca6118417544267fb700672ec9d37
DESCIFRADO: Seguridad informática con AES-256
```

Figure 9: Ejecución de Script Python del Ejercicio 6

Responder:

- ¿Por qué la clave AES-256 tiene exactamente 32 bytes?

Porque AES-256 utiliza una clave de 256 bits y cada byte equivale a 8 bits, por lo que $256 \div 8 = 32$ bytes.

- ¿Qué función cumple el nonce en el proceso de cifrado?

El nonce introduce aleatoriedad en el cifrado para que un mismo mensaje cifrado varias veces produzca resultados diferentes.

- ¿Qué ocurre si se utiliza la misma clave y nonce en dos mensajes diferentes?

Se compromete la seguridad del cifrado, ya que un atacante podría obtener información sobre los mensajes y facilitar ataques criptográficos.

- ¿Por qué AES se considera un algoritmo simétrico?

Porque utiliza la misma clave tanto para cifrar como para descifrar la información.

- ¿Qué ventaja tiene el modo GCM frente a CBC?

GCM proporciona confidencialidad e integridad de los datos, mientras que CBC solo proporciona confidencialidad y requiere mecanismos adicionales para verificar la integridad.

Actividad de modificación y mejoramiento del algoritmo propuesto

Objetivo: Modificar el programa base para analizar la robustez del cifrado AES-256-GCM y comprender el impacto de cambios en sus componentes.

Actividades propuestas

Actividad 1: Manipulación del ciphertext

Modificar el código agregando:

```
ciphertext = bytearray(ciphertext)
ciphertext[0] ^= 1
ciphertext = bytes(ciphertext)
```

```
(env) msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/$ python ej7.py
MENSAJE ORIGINAL: Seguridad informática con AES-256
CLAVE: 5f271d74f9e1ee2245a724d0e0fb5c72685f3bbe4726b445idf7b82d811c36
NONCE: 428708124343546fb6fce349
CIPHERDATA: 983137467c436694b4d813280a30170603eb47f1b62fc1b1984e623989b6441424ca359f89a8860c364fd6f3a0b54f461c0
Traceback (most recent call last):
  File "/home/msosa/Documents/ESPE/seguridad-30808/P2/Laboratorios/3/ej7.py", line 28, in <module>
    texto_descifrado = aesgcm.decrypt(nonce, ciphertext, None)
  File "/usr/lib64/python3.10/site-packages/cryptography/hazmat/primitives/aes_gcm.py", line 100, in decrypt
    raise ValueError(
ValueError: Invalid tag
```

Figure 10: Ejecución de Script Python de Ejercicio 7

Pregunta

¿Qué ocurre al intentar descifrar el mensaje modificado?

Genera un error proveniente de la librería instalada al no conocer cómo poder descifrar el mensaje.

Investigación

Investigue los modos de operación ECB (Electronic Codebook), CBC (Cipher Block Chaining), CFB (Cipher Feedback), OFB (Output Feedback) **GCM (Galois/Counter Mode)** en AES; cifre un archivo de texto utilizando al menos dos de ellos con OpenSSL en Kali Linux; compare los resultados visualmente con hexdump; y redacte una breve reflexión sobre las diferencias observadas y su impacto en la seguridad de la información.

Modo	Descripción	Ventajas	Desventajas	Uso recomendado
ECB (Electronic Codebook)	Cifra cada bloque de forma independiente.	Simple y rápido.	Bloques iguales producen cifrados iguales, revelando patrones.	No recomendado para datos sensibles.
CBC (Cipher Block Chaining)	Cada bloque depende del bloque anterior mediante un IV.	Oculto patrones y mejora la seguridad respecto a ECB.	Requiere IV y no permite paralelismo completo en el cifrado.	Archivos y almacenamiento general.
CFB (Cipher Feedback)	Convierte un cifrador de bloques en uno de flujo mediante retroalimentación.	Permite cifrar datos de tamaño variable.	Un error puede propagarse temporalmente.	Comunicaciones en tiempo real.
OFB (Output Feedback)	Genera un flujo de claves independiente del texto plano.	Los errores no se propagan.	Reutilizar IV compromete la seguridad.	Canales con ruido o transmisión de datos.
GCM (Galois/Counter Mode)	Combina cifrado en modo contador con autenticación.	Confidencialidad, integridad y alto rendimiento.	Requiere manejo correcto de nonces.	Aplicaciones modernas, TLS, VPN y almacenamiento seguro.

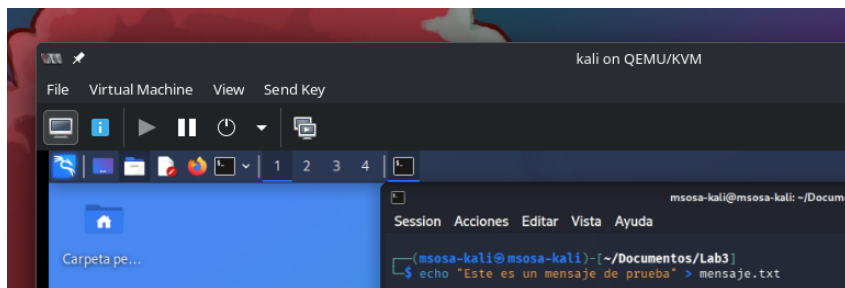


Figure 11: Creación del Archivo del Ejercicio 8

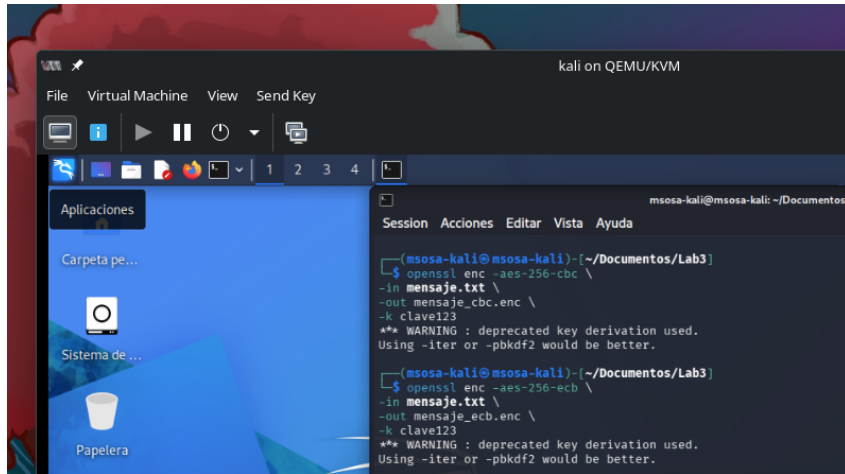


Figure 12: Encriptado del Archivo mediante Diversos Algoritmos

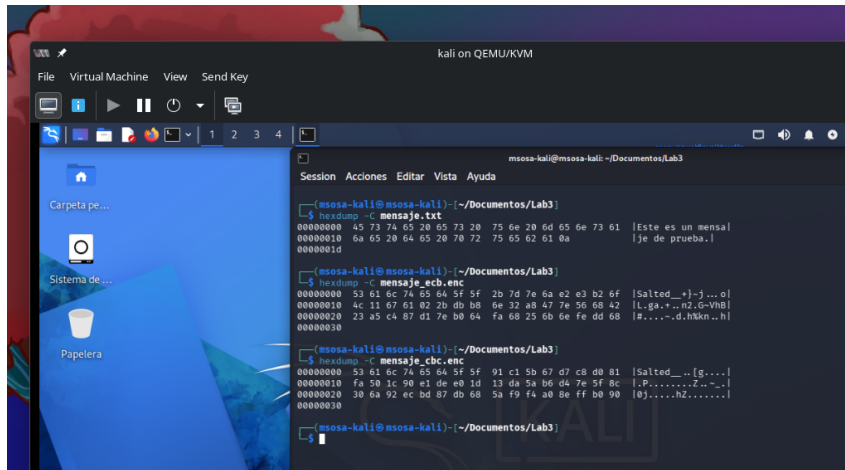


Figure 13: Revisión de Archivos Encriptados mediante hexdump

Al comparar los archivos cifrados mediante `hexdump`, se observa que todos los modos generan datos aparentemente aleatorios, pero ECB puede revelar patrones cuando existen bloques repetidos, mientras que CBC y GCM producen resultados más seguros al incorporar información adicional como IV o nonce. GCM destaca porque además de cifrar los datos garantiza su integridad, evitando modificaciones no autorizadas, por lo que es uno de los modos más utilizados actualmente en aplicaciones y protocolos modernos.

Conclusiones:

1. Esta práctica ha proporcionado una introducción a la criptografía simétrica utilizando el algoritmo AES. A través de la utilización de **OpenSSL**, los estudiantes pudieron cifrar y descifrar un mensaje, aprendiendo los conceptos clave detrás de los **métodos de cifrado** y los **modos de operación** utilizados en criptografía moderna. Además, se destacó la importancia de la **gestión de claves** en criptografía simétrica.
2. La criptografía simétrica es eficiente en términos de velocidad, pero su principal vulnerabilidad es la **gestión de claves**. Ambas partes que cifran y descifran deben compartir la misma clave, lo que puede generar problemas de seguridad si la clave es interceptada.

3. AES es un algoritmo seguro, pero su seguridad depende de su correcta implementación.
4. El manejo incorrecto de claves puede comprometer completamente el sistema.

Referencias Bibliográficas:

- Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson.
- Ferguson, N., Schneier, B., & Kohno, T. (2010). *Cryptography Engineering: Design Principles and Practical Applications*. Wiley.
- OpenSSL Project. (n.d.). *OpenSSL Documentation*. Retrieved from <https://www.openssl.org/docs/>
- Schneier, B. (1996). *Applied Cryptography: Protocols, Algorithms, and Source Code in C* (2nd ed.). Wiley.



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de la Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: June 12, 2026

Contents

Objetivos de Aprendizaje	3
Marco Teórico	3
Entorno de laboratorio	4
Desarrollo	4
Generar claves RSA	4
Crear un archivo de texto de prueba	5
Encriptar el archivo con la clave pública	5
Desencriptar el archivo con la clave privada	6
Verificación de integridad del archivo	6
Conclusiones	8
Referencias Bibliográficas	8

List of Figures

1	Protocolos y Algoritmos Asimétricos y su relación con los modelos ISO/OSI y TCP/IP.	3
2	Generación de Clave Privada RSA	4
3	Generación de Clave Pública RSA	4
4	Contenido de Clave Privada	5
5	Contenido de Clave Pública	5
6	Creación de Archivo de Texto de Prueba	5
7	Encriptación de Archivo con Clave Pública	6
8	Desencriptación de Archivo con Clave Privada	6
9	Comparación de Archivo Encriptado y Original	6
10	Ejecución de Script Python de Ejercicio 6	7

List of Tables

Objetivos de Aprendizaje

Al finalizar esta práctica, usted será capaz de:

- Comprender los fundamentos teóricos del algoritmo de encriptación asimétrica RSA.
- Generar y gestionar claves públicas y privadas RSA utilizando OpenSSL.
- Encriptar y desencriptar archivos mediante RSA.
- Analizar el funcionamiento del algoritmo RSA original y su implementación básica en Python.
- Aplicar pruebas básicas de seguridad y verificación sobre archivos cifrados.
- Identificar casos prácticos de uso de la encriptación asimétrica en sistemas reales.

Marco Teórico

La encriptación asimétrica es un mecanismo criptográfico que utiliza un par de claves: **una pública** (para cifrar) y **una privada** (para descifrar). Uno de los algoritmos más conocidos en esta categoría es **RSA** (Rivest-Shamir-Adleman), desarrollado en 1977 (véase Figura 1).

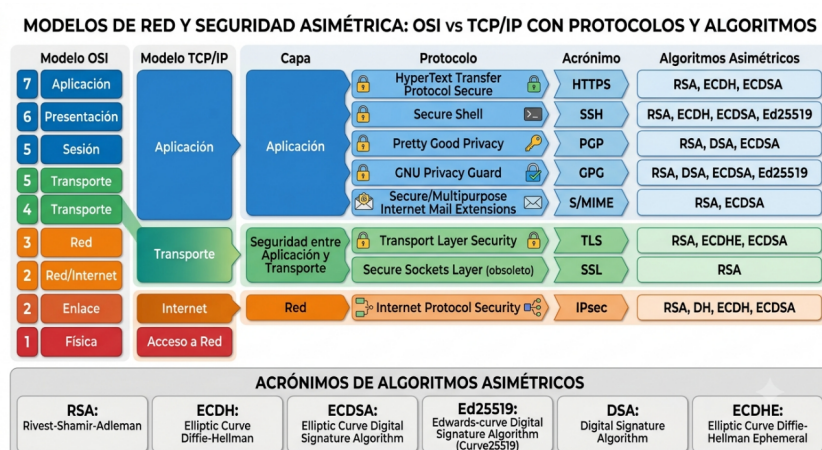


Figure 1: Protocolos y Algoritmos Asimétricos y su relación con los modelos ISO/OSI y TCP/IP.

RSA basa su seguridad en la dificultad matemática de factorizar números grandes (producto de dos números primos). Su uso se extiende desde la protección de comunicaciones en Internet (TLS/SSL) hasta la firma digital y almacenamiento seguro de datos.

OpenSSL es una herramienta de línea de comandos ampliamente usada para gestionar certificados digitales, cifrar información, generar claves RSA, y más. Permite implementar en la práctica muchos conceptos teóricos de la criptografía.

Según Stallings (2020), RSA es fundamental para entender los mecanismos de cifrado seguro y autenticación digital, gracias a su estructura basada en la teoría de números.

Entorno de laboratorio

- Máquina virtual con **Kali Linux** (como auditor de seguridad)
- Máquina virtual con **Ubuntu Desktop** (como cliente)
- Máquina virtual con **Ubuntu Server** (como servidor de pruebas)
- Herramienta: **OpenSSL**

Desarrollo

Generar claves RSA

En Ubuntu Desktop o Server:

```
openssl genpkey -algorithm RSA -out clave_privada.pem -pkeyopt rsa_keygen_bits:2048
```

```
openssl rsa -pubout -in clave_privada.pem -out clave_publica.pem
```

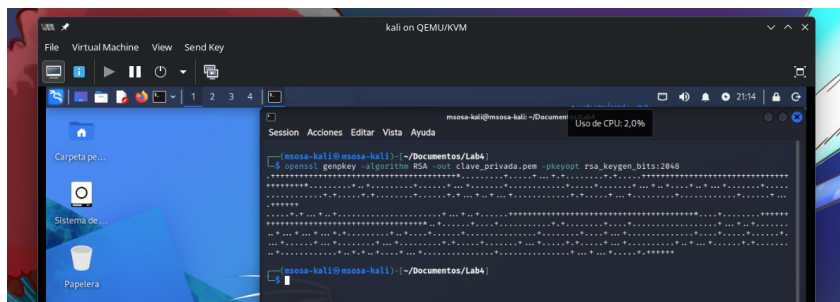


Figure 2: Generación de Clave Privada RSA

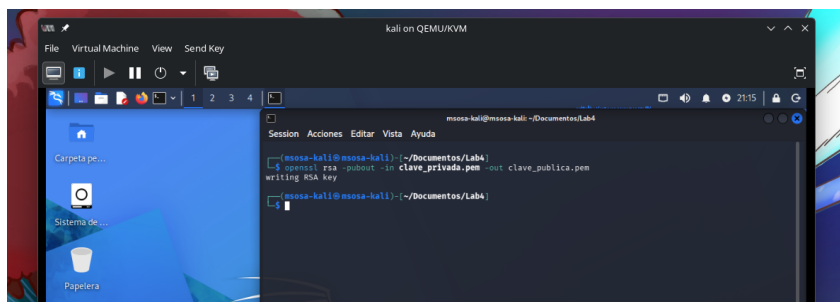


Figure 3: Generación de Clave Pública RSA

Verifique las claves:

```
cat clave_privada.pem
```

```
cat clave_publica.pem
```

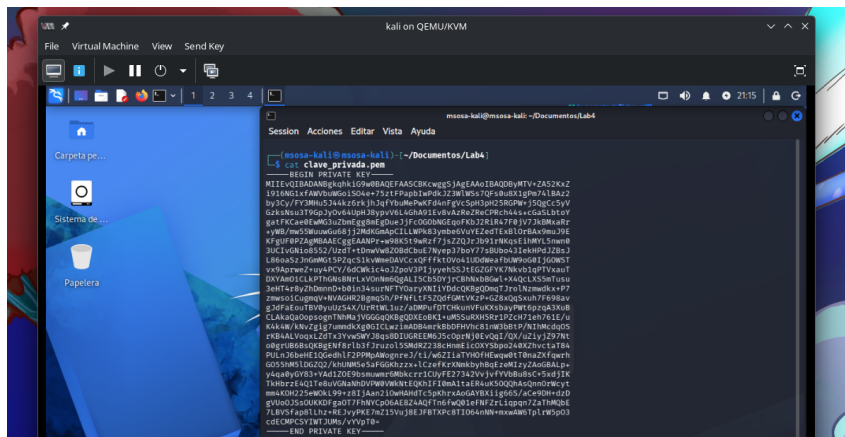


Figure 4: Contenido de Clave Privada

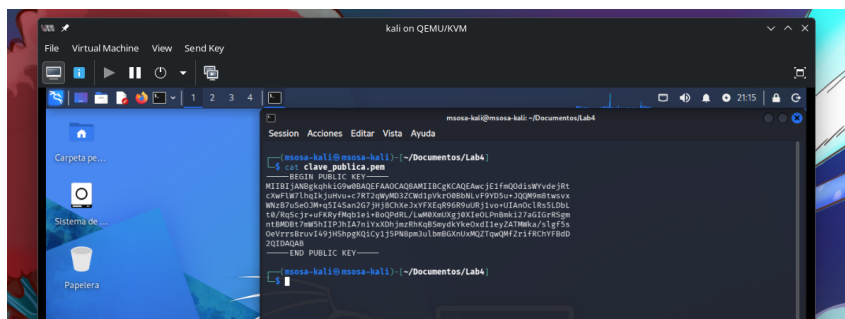


Figure 5: Contenido de Clave Pública

Crear un archivo de texto de prueba

```
echo "Confidential information for encryption. Linux is recommended for enhanced security."
```

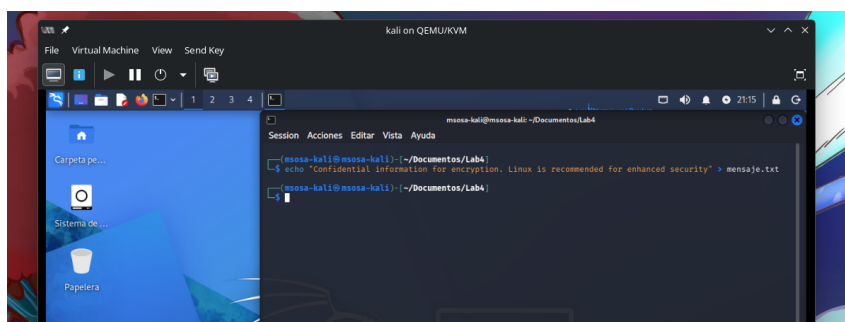


Figure 6: Creación de Archivo de Texto de Prueba

Encriptar el archivo con la clave pública

```
openssl rsautl -encrypt -inkey clave_publica.pem -pubin -in mensaje.txt -out mensaje.encrypted
```

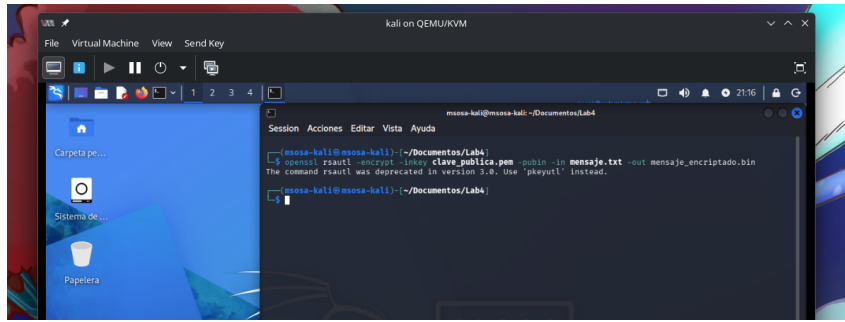


Figure 7: Encriptación de Archivo con Clave Pública

Desencriptar el archivo con la clave privada

`openssl rsautl -decrypt -inkey clave_privada.pem -in mensaje_encryptado.bin -out mensaje_desencriptado.txt`

Verifique el contenido:

`cat mensaje_desencriptado.txt`

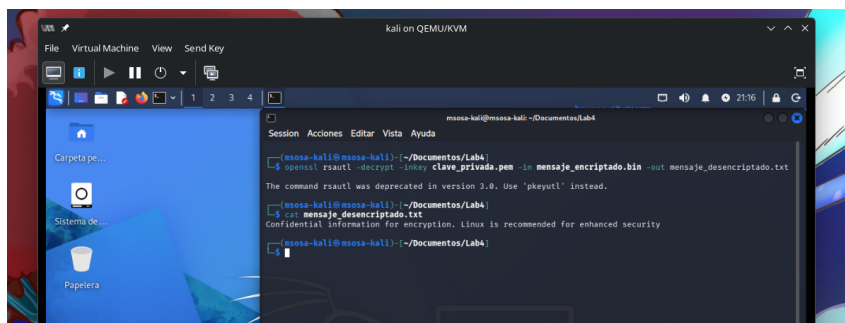


Figure 8: Desencriptación de Archivo con Clave Privada

Verificación de integridad del archivo

Compare los archivos original y desencriptado:

`diff mensaje.txt mensaje_desencriptado.txt`

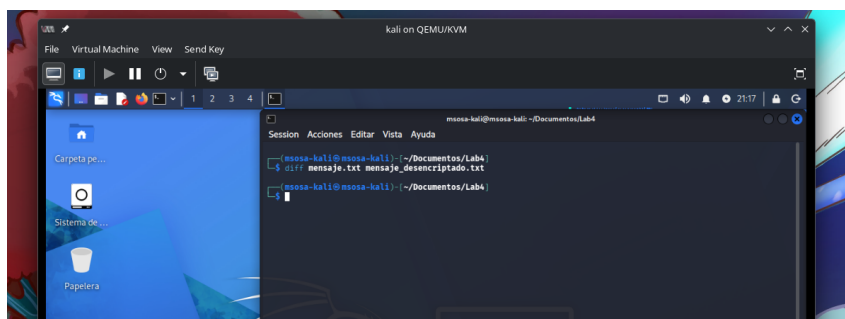


Figure 9: Comparación de Archivo Encriptado y Original

Actividad complementaria en clase (30 minutos)

Analizar e implementar RSA en Python

Los estudiantes deberán analizar e implementar un script básico del algoritmo RSA en Python (sin usar librerías externas como cryptography). Se les entrega el siguiente código base:

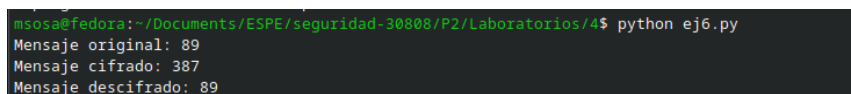
```
# RSA básico en Python (solo para fines educativos)
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def modinv(a, m):
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

# Parámetros básicos (pequeños por simplicidad)
p = 17
q = 23
n = p * q
phi = (p-1)*(q-1)
e = 3
d = modinv(e, phi)

mensaje = 89
cifrado = pow(mensaje, e, n)
descifrado = pow(cifrado, d, n)

print(f"Mensaje original: {mensaje}")
print(f"Mensaje cifrado: {cifrado}")
print(f"Mensaje descifrado: {descifrado}")
```



```
msosa@fedora:~/Documents/ESPE/seguridad-30808/P2/Laboratorios/4$ python ej6.py
Mensaje original: 89
Mensaje cifrado: 387
Mensaje descifrado: 89
```

Figure 10: Ejecución de Script Python de Ejercicio 6

Preguntas de análisis (discusión en parejas):

1. ¿Qué rol cumple e y d en el cifrado y descifrado?

e es el exponente público utilizado para cifrar el mensaje, mientras que d es el exponente privado utilizado para descifrarlo.

2. ¿Por qué se usa el módulo n?

El módulo n define el rango de los cálculos matemáticos de RSA y permite que el cifrado y descifrado funcionen correctamente.

3. ¿Qué implicaciones tendría usar claves más grandes en este script?

Claves más grandes aumentan la seguridad del sistema, pero también requieren más tiempo y recursos para realizar los cálculos.

4. ¿Cuáles son las principales aplicaciones?

RSA se utiliza principalmente para intercambio seguro de claves, firmas digitales, certificados digitales y comunicaciones seguras en protocolos como HTTPS.

Conclusiones

1. La práctica permitió comprender de manera teórica y práctica el funcionamiento del algoritmo RSA, identificando el papel de las claves pública y privada dentro de un esquema de cifrado asimétrico.
2. Mediante el uso de OpenSSL se logró generar, administrar y verificar pares de claves RSA de 2048 bits, evidenciando la importancia de utilizar herramientas especializadas para la implementación segura de mecanismos criptográficos.
3. El proceso de cifrado y descifrado de archivos demostró que únicamente la clave privada correspondiente puede recuperar la información protegida con la clave pública, garantizando la confidencialidad de los datos transmitidos o almacenados.

Referencias Bibliográficas

1. Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson.
2. Ferguson, N., Schneier, B., & Kohno, T. (2010). *Cryptography Engineering: Design Principles and Practical Applications*. Wiley.
3. OpenSSL Project. (n.d.). *OpenSSL Documentation*. Retrieved from <https://www.openssl.org/docs/>
4. Schneier, B. (1996). *Applied Cryptography: Protocols, Algorithms, and Source Code in C* (2nd ed.). Wiley.
5. NIST. (2001). *Announcing the Advanced Encryption Standard (AES)*. Federal Information Processing Standards Publication 197. U.S. National Institute of Standards and Technology.
6. Upadhyay, D., Sampalli, S., & Plourde, B. (2020). Vulnerabilities' assessment and mitigation strategies for the small Linux server, Onion Omega2. *Electronics*, 9(6), 967.



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de la Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: June 12, 2026

Contents

Topología de Prueba	4
Objetivos de Aprendizaje	4
Marco Teórico	4
Certificado digital	4
Desarrollo	5
Creación y Gestión de Certificados digitales en Kali Linux	5
Comandos básicos de administración y visualización de certificados digitales .	7
Aplicación de criptografía simétrica y asimétrica en el contexto de certificados digitales (PKI)	9
Generación y Verificación de Firmas Digitales	13
Conclusiones Esperadas de la Práctica	15
Referencias Bibliográficas	16

List of Figures

1	Generación de Clave Privada	6
2	Generación del CSR	6
3	Auto-Firma del Certificado	6
4	Certificado Completo	7
5	CA Emisora	8
6	Propietario	8
7	Clave Pública	8
8	Huella Digital	9
9	Verificar Firma	9
10	Ver CSR	9
11	Creación del Mensaje a Cifrar	11
12	Generación de Clave Privada RSA	11

13	Extracción de Clave Pública	11
14	Visualización del Certificado y Verificación de Identidad	12
15	Cifrado del Mensaje con Clave Pública	12
16	Creación del Mensaje para Cifrado Simétrico	12
17	Cifrado Simétrico del Mensaje con AES	13
18	Descifrado Simétrico del Mensaje	13
19	Documento a Firmar	14
20	Generación de la Firma	14
21	Verificación de la Firma	15
22	Prueba de Integridad	15

List of Tables

1	Procedimiento para la generación de certificados digitales autofirmados con OpenSSL en Kali Linux.	5
2	Comandos OpenSSL para inspección de certificados X.509 y elementos de PKI.	7
3	Aplicación de criptografía simétrica y asimétrica en el contexto de certificados digitales en Kali.	10
4	Proceso de firma digital y validación de integridad con certificados X.509.	13

Topología de Prueba

Entorno Virtual de Red: Kali Linux, Ubuntu Desktop, Ubuntu Server, Metasploitable2 y OpenSSL.

Objetivos de Aprendizaje

Objetivo 1: Comprensión y Gestión de la PKI (Infraestructura de Clave Pública): Al finalizar la práctica, el estudiante será capaz de gestionar los componentes fundamentales de la Infraestructura de Clave Pública (PKI) simulada, aplicando comandos de OpenSSL en Kali Linux para la generación de claves privadas, la creación de Solicitudes de Firma de Certificado (CSR) y el auto-firmado de certificados digitales (self-signed certificates).

Objetivo 2: Aplicación de Algoritmos Criptográficos (Cifrado y Descifrado): Demostrar la habilidad práctica para implementar y verificar el cifrado y descifrado de datos sensibles, utilizando la herramienta OpenSSL con diversos algoritmos criptográficos (como AES y RSA), y contrastar la seguridad y eficiencia de los esquemas de criptografía simétrica y asimétrica en un entorno de pruebas.

Objetivo 3: Verificación de Integridad y Autenticidad (Firmas Digitales): Aplicar los mecanismos de seguridad criptográfica para garantizar la integridad y autenticidad de archivos. Esto incluye generar una firma digital sobre un archivo mediante la clave privada en OpenSSL y, posteriormente, verificar la validez de dicha firma utilizando el certificado digital correspondiente, identificando cualquier manipulación del contenido original.

Marco Teórico

Certificado digital

Un certificado digital es un archivo o contraseña electrónica que demuestra la autenticidad de un dispositivo, servidor o usuario mediante el uso de criptografía de la infraestructura de clave pública (public key infrastructure, PKI).

OpenSSL es una herramienta de software de código abierto ampliamente utilizada en sistemas Linux (y otros sistemas operativos) para la implementación de funciones criptográficas. Ofrece una biblioteca robusta que proporciona un conjunto de algoritmos de cifrado, funciones de seguridad y utilidades para la gestión de certificados SSL/TLS, la generación de claves y la creación de firmas digitales. Entre sus principales usos se tiene [3] :

- OpenSSL permite realizar cifrado y descifrado de datos utilizando una variedad de algoritmos criptográficos, como **AES**, **DES**, **RSA**, **Blowfish**, entre otros.
- OpenSSL permite generar claves públicas y privadas para criptografía simétrica y asimétrica.
- OpenSSL permite generar certificados digitales utilizados en protocolos como **SSL/TLS** para asegurar la comunicación en redes

Desarrollo

Creación y Gestión de Certificados digitales en Kali Linux

En este apartado se presenta el procedimiento básico para la creación y gestión de certificados digitales en Kali Linux utilizando OpenSSL. Se detalla, de forma práctica, la generación de la clave privada, la creación de la solicitud de firma de certificado (CSR) y la emisión de un certificado autofirmado. Este flujo permite comprender los elementos fundamentales de una Infraestructura de Clave Pública (PKI) y la relación entre la clave privada, la identidad digital y el certificado resultante. A continuación, se muestra el conjunto de pasos y comandos utilizados en el proceso.

Table 1: Procedimiento para la generación de certificados digitales autofirmados con OpenSSL en Kali Linux.

Tarea	Comando en Kali Linux	Descripción de la Acción
1.1. Generación de Clave Privada	<code>openssl genrsa -out server_private.key 2048</code>	Guarda la clave privada generada por genrsa en el archivo <code>server_private.key</code> de 2048 bits para el servidor de prueba.
1.2. Generación del CSR	<code>openssl req -new -key server_private.key -out server.csr</code>	Crea la Solicitud de Firma de Certificado (CSR), ingresando los datos de identidad. req: Trabaja con solicitudes de certificados (CSR) y certificados X.509. -new: Indica que se va a crear una nueva CSR. -key <code>server_private.key</code> : Usa la clave privada creada en el paso anterior para firmar la solicitud. -out <code>server.csr</code> : Guarda la CSR generada en <code>server.csr</code> .
1.3. Auto-Firma del Certificado	<code>openssl x509 -req -days 365 -in server.csr -signkey server_private.key -out server_certificate.crt</code>	x509: Trabaja con certificados X.509. -req: Indica que la entrada es una CSR. -in <code>server.csr</code> : CSR de entrada. -signkey <code>server_private.key</code> : Firma el certificado con esta clave privada. -out <code>server_certificate.crt</code> : Guarda el certificado generado. Utiliza la clave privada local para firmar el CSR y crear un certificado digital válido por un año.

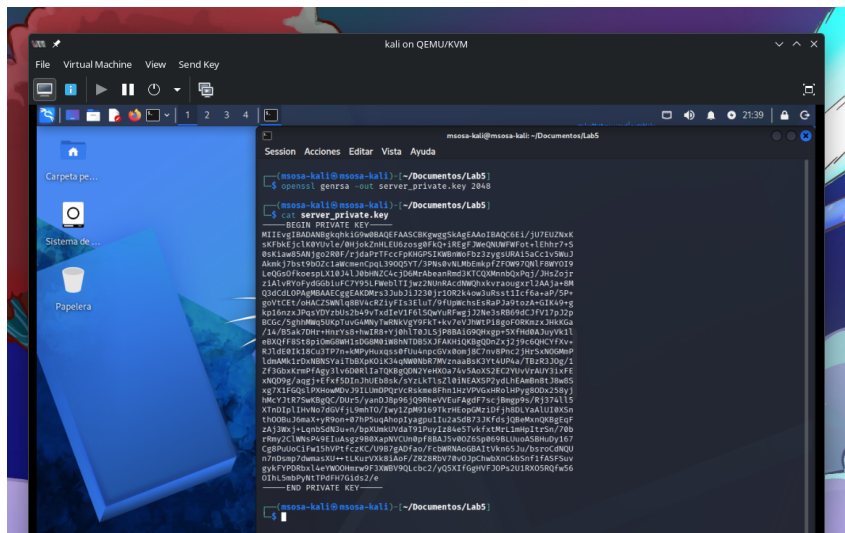


Figure 1: Generación de Clave Privada

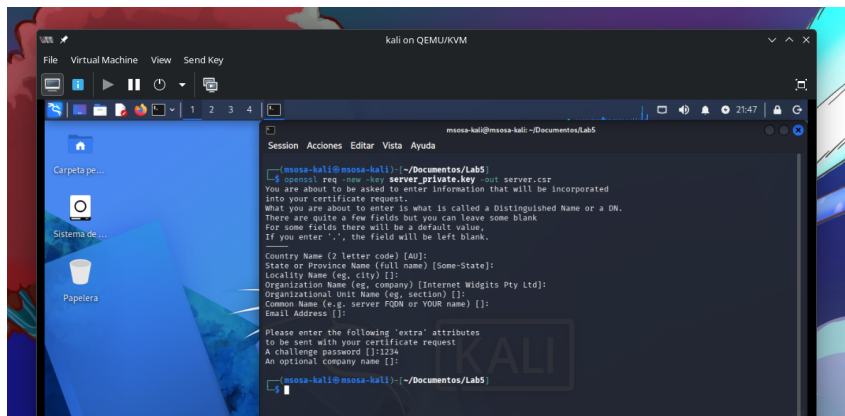


Figure 2: Generación del CSR

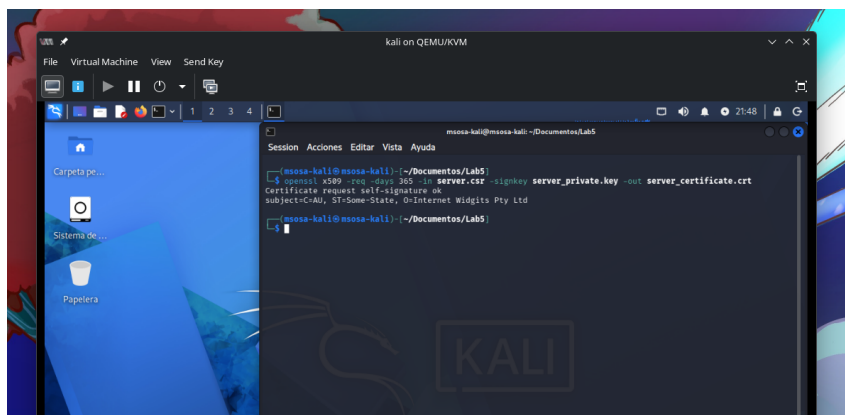


Figure 3: Auto-Firma del Certificado

Comandos básicos de administración y visualización de certificados digitales

Una vez generado un certificado digital, es importante conocer su estructura y los elementos que garantizan su autenticidad e integridad. Mediante herramientas como OpenSSL es posible inspeccionar la información contenida en un certificado X.509, incluyendo el propietario (Subject), la entidad emisora o Autoridad Certificadora (Issuer), la clave pública asociada, el período de validez y la firma digital que permite verificar su autenticidad. La visualización de estos componentes facilita la comprensión del funcionamiento de una Infraestructura de Clave Pública (PKI) y de los mecanismos de confianza utilizados en la seguridad informática moderna. A continuación, en la Tabla 2, se presentan algunos comandos básicos para examinar y validar certificados digitales en sistemas Linux.

Table 2: Comandos OpenSSL para inspección de certificados X.509 y elementos de PKI.

Componente	Comando
Certificado completo	<code>openssl x509 -in cert.crt -text -noout</code>
CA emisora	<code>openssl x509 -issuer -noout -in cert.crt</code>
Propietario	<code>openssl x509 -subject -noout -in cert.crt</code>
Clave pública	<code>openssl x509 -pubkey -noout -in cert.crt</code>
Huella digital	<code>openssl x509 -fingerprint -sha256 -noout -in cert.crt</code>
Verificar firma	<code>openssl verify -CAfile cert.crt cert.crt</code>
Ver cadena PKI remota	<code>openssl s_client -connect sitio:443 -showcerts</code>
Ver CSR	<code>openssl req -in server.csr -text -noout</code>
Verificar clave a certificado	Comparar el modulus de la clave privada y del certificado para comprobar que pertenecen al mismo par.

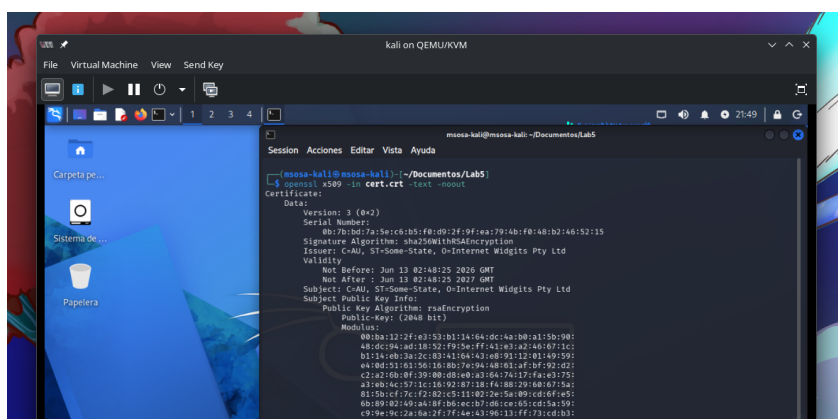


Figure 4: Certificado Completo

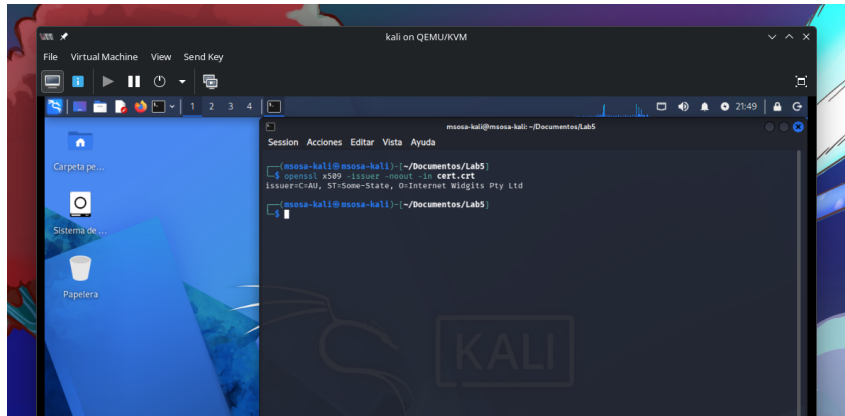


Figure 5: CA Emisora

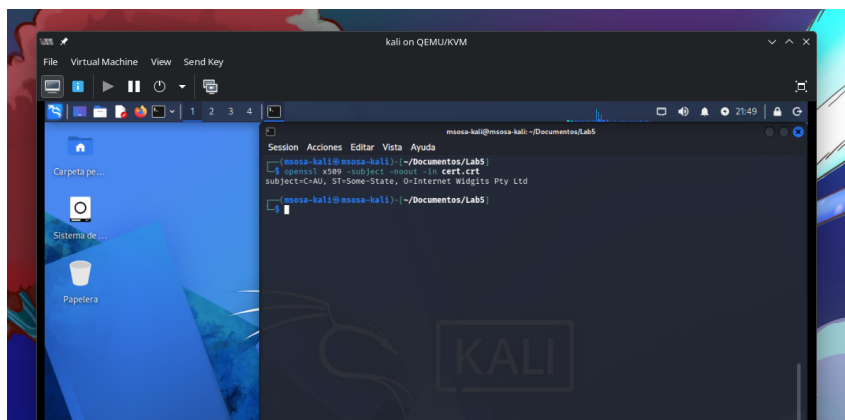


Figure 6: Proprietario

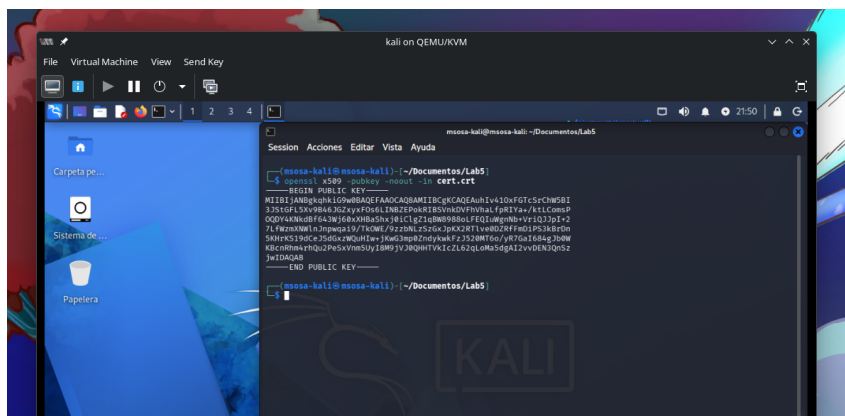


Figure 7: Clave Pública

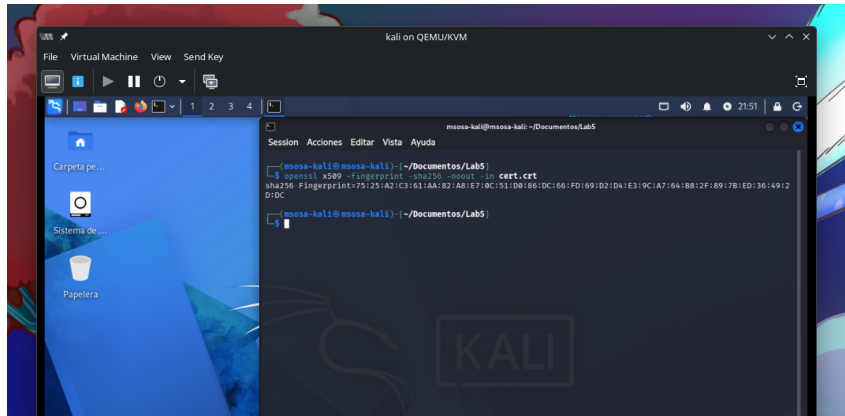


Figure 8: Huella Digital

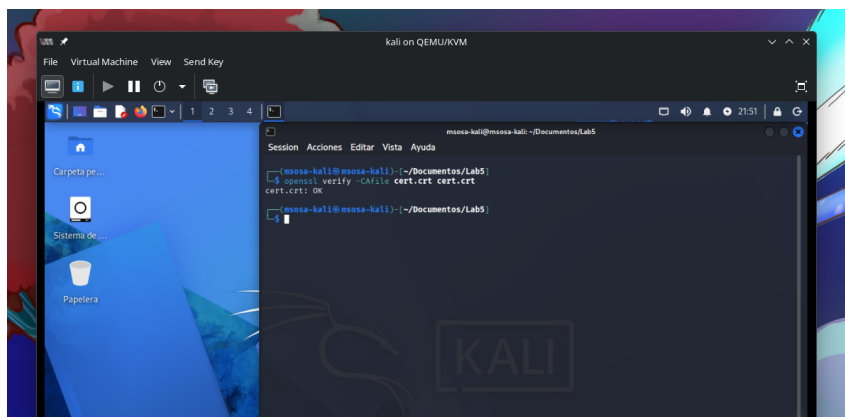


Figure 9: Verificar Firma

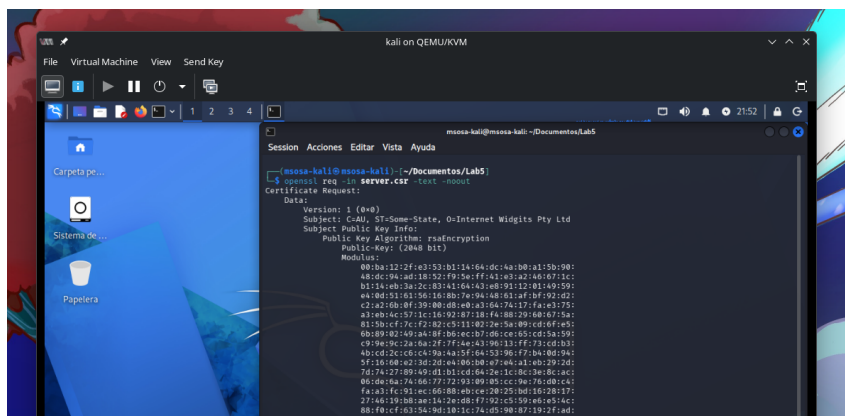


Figure 10: Ver CSR

Aplicación de criptografía simétrica y asimétrica en el contexto de certificados digitales (PKI)

En este apartado se extiende la práctica previa de generación y visualización de certificados digitales en Kali Linux, incorporando su aplicación en escenarios de cifrado de información.

A partir de los elementos fundamentales de una Infraestructura de Clave Pública (PKI), como la clave privada, la clave pública y el certificado X.509, se implementan mecanismos de protección de datos mediante criptografía simétrica y asimétrica.

La Tabla 3 presenta un flujo integrado donde se relaciona la creación del mensaje, el uso de claves derivadas del proceso de certificación y la aplicación de algoritmos de cifrado. De esta manera, el estudiante puede observar cómo los certificados digitales no solo permiten la autenticación de identidades, sino que también constituyen la base para el intercambio seguro de información en entornos Linux. [OpenSSL, 2024].

Table 3: Aplicación de criptografía simétrica y asimétrica en el contexto de certificados digitales en Kali.

Actividad	Comando en Kali Linux	Tipo de Criptografía
3.1. Creación del mensaje a cifrar	echo "Mensaje Asimétrico" > asymmetric_message.txt	Asimétrica (dato de entrada)
3.2. Generación de clave privada RSA (base del certificado)	openssl genrsa -out server_private.key 2048	Asimétrica (clave privada del sistema PKI)
3.3. Extracción de clave pública desde la clave privada del certificado	openssl rsa -in server_private.key -pubout -out server_public.key	Asimétrica (clave pública asociada al certificado)
3.4. Visualización del certificado y verificación de identidad (PKI)	openssl x509 -in server_certificate.crt -text -noout	PKI / X.509 (inspección del certificado)
3.5. Cifrado del mensaje usando clave pública del certificado	openssl pkeyutl -encrypt -pubin -inkey server_public.key -in asymmetric_message.txt -out asymmetric_message.enc	Asimétrica (cifrado con clave pública)
3.6. Creación del mensaje para cifrado simétrico	echo "Credencial Secreta: 1A2b3C4d" > confidential.txt	Simétrica (dato de entrada)
3.7. Cifrado simétrico del mensaje (AES)	openssl enc -aes-256-cbc -salt -in confidential.txt -out confidential.enc	Simétrica (cifrado con clave compartida)
3.8. Descifrado simétrico del mensaje (AES)	openssl enc -d -aes-256-cbc -in confidential.enc -out decrypted.txt	Simétrica (verificación de integridad)

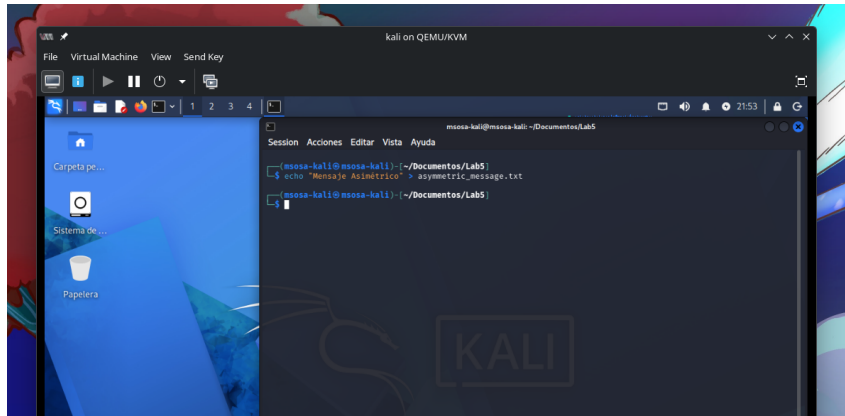


Figure 11: Creación del Mensaje a Cifrar

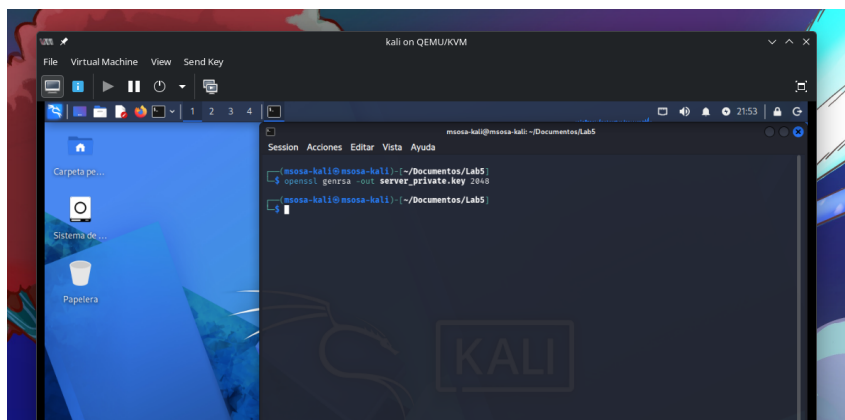


Figure 12: Generación de Clave Privada RSA

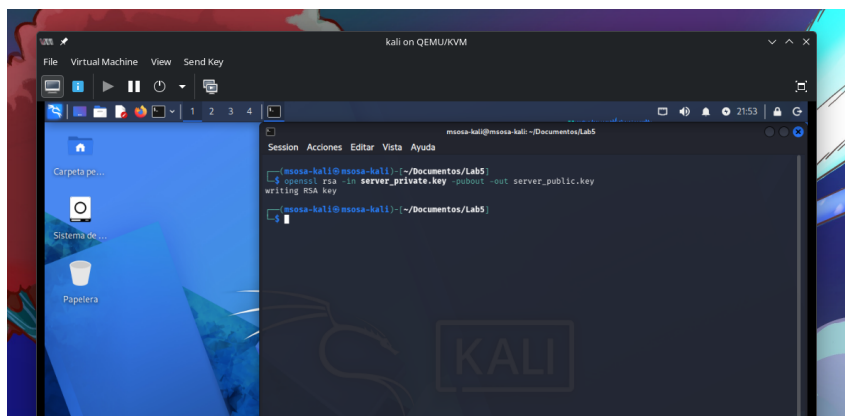
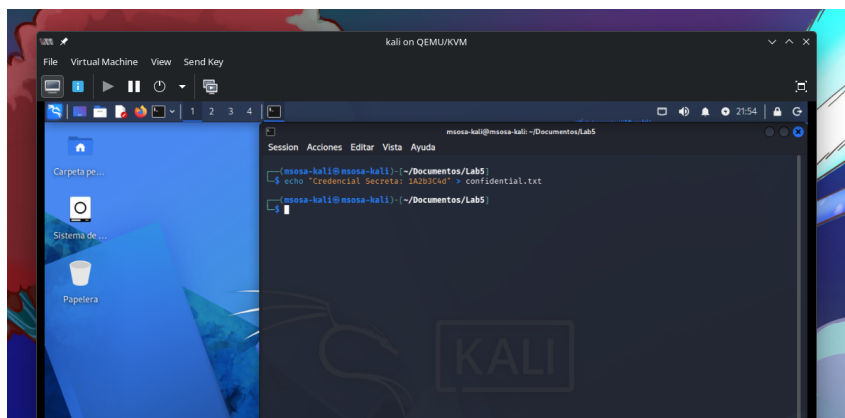
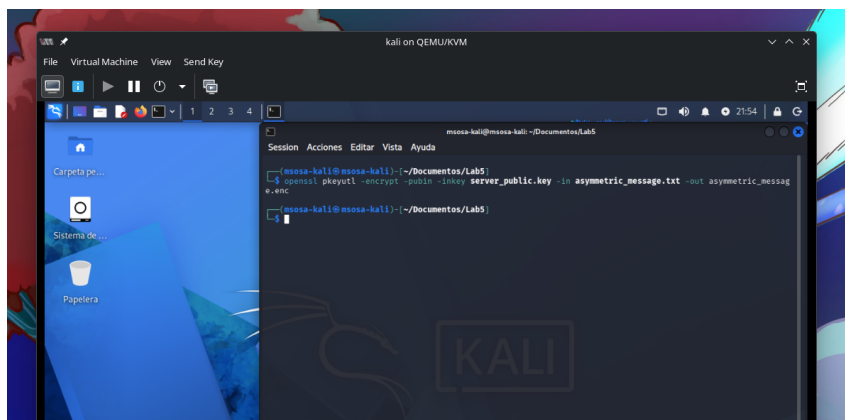
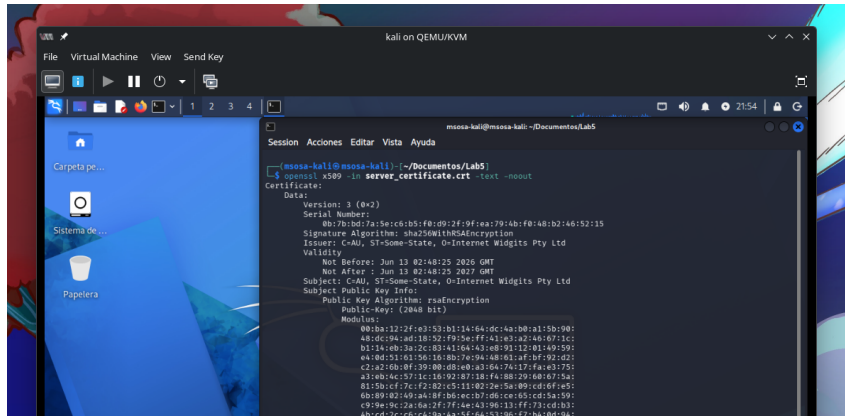


Figure 13: Extracción de Clave Pública



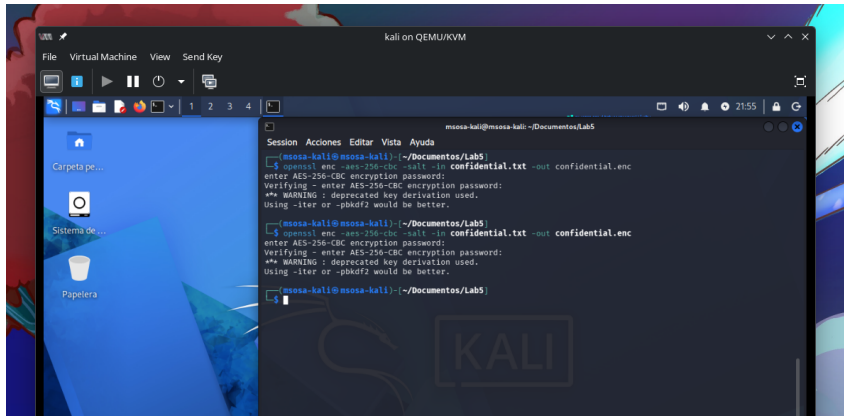


Figure 17: Cifrado Simétrico del Mensaje con AES

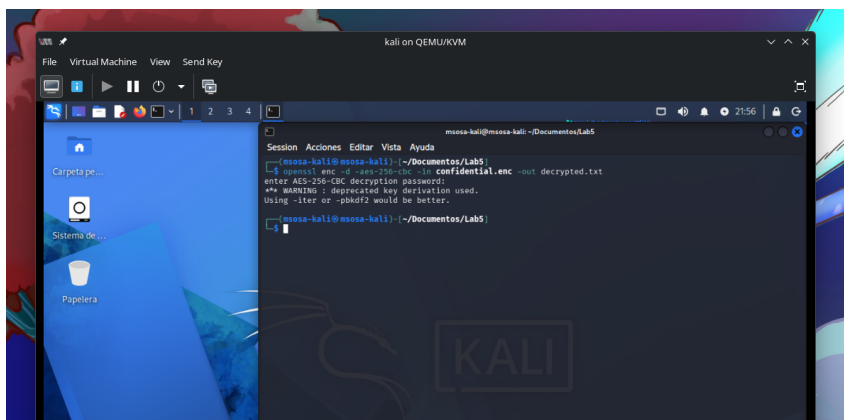


Figure 18: Descifrado Simétrico del Mensaje

Generación y Verificación de Firmas Digitales

En este apartado se presenta la aplicación práctica de la firma digital como mecanismo fundamental dentro de una Infraestructura de Clave Pública (PKI). A diferencia del cifrado, cuyo objetivo es proteger la confidencialidad de la información, la firma digital permite garantizar la integridad, autenticidad y no repudio de los datos.

La Tabla 4 muestra el proceso completo de generación de una firma digital utilizando la clave privada asociada al certificado digital previamente creado, así como su verificación mediante la clave pública contenida en dicho certificado. Además, se incluye una prueba de modificación del archivo para evidenciar cómo cualquier alteración invalida la firma, demostrando así la importancia de la integridad en la seguridad de la información.

Table 4: Proceso de firma digital y validación de integridad con certificados X.509.

Tarea	Comando en Kali Linux	Propósito de la Criptografía
4.1. Documento a Firmar	echo "Versión 1.0.0: Código Auditado y Aprobado" > code_release.txt	Crear el archivo de prueba.

Tarea	Comando en Kali Linux	Propósito de la Criptografía
4.2. Generación de la Firma	<code>openssl dgst -sha256 -sign server_private.key -out code_release.sig code_release.txt</code>	Crea la firma digital cifrando el hash SHA-256 del archivo con la clave privada.
4.3. Verificación de la Firma	<code>openssl dgst -sha256 -verify server_public.key -signature code_release.sig code_release.txt</code>	Utiliza la clave pública asociada al certificado para verificar la firma y comparar el hash calculado con el hash firmado.
4.4. Prueba de Integridad (Fallo)	(Modificar manualmente el archivo <code>code_release.txt</code>) <code>openssl dgst -sha256 -verify server_public.key -signature code_release.sig code_release.txt</code>	Debe resultar en “Verification Failure”, demostrando que la integridad del documento fue alterada.

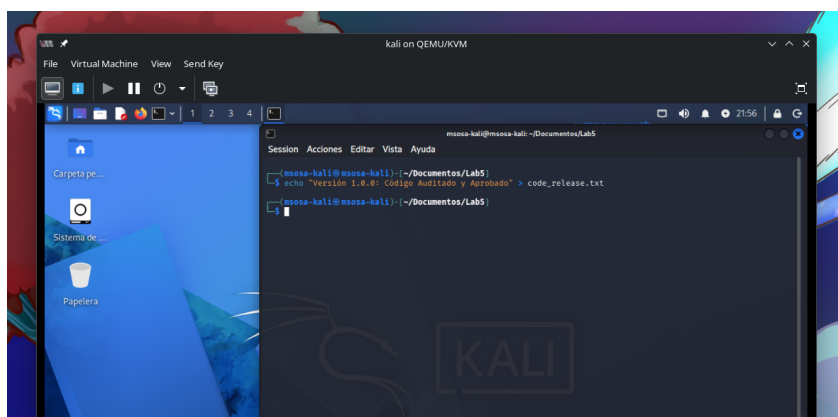


Figure 19: Documento a Firmar

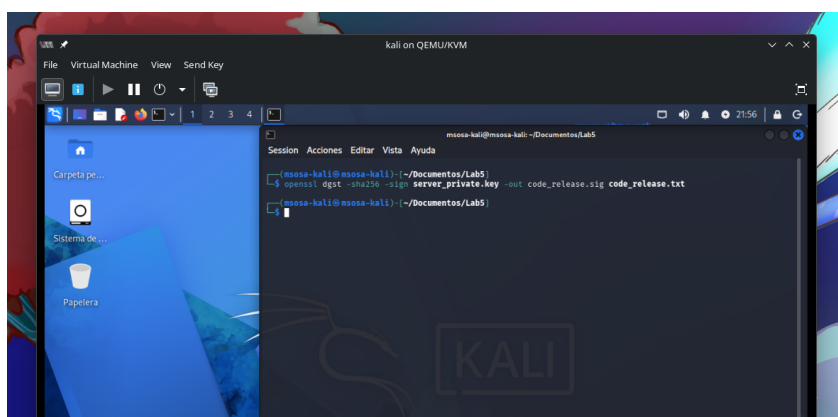


Figure 20: Generación de la Firma

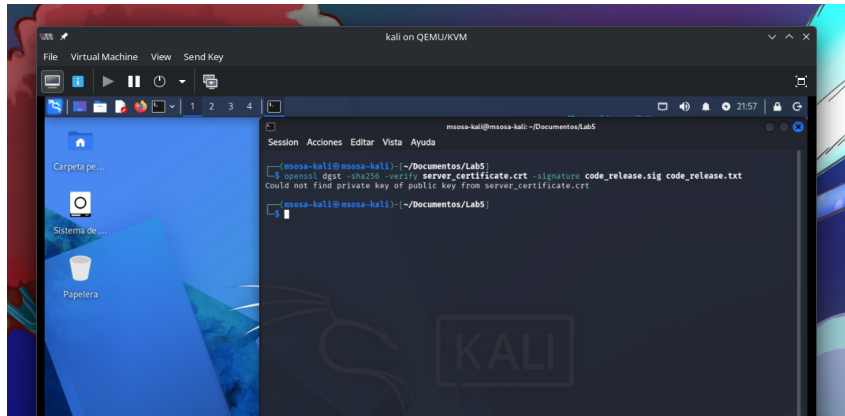


Figure 21: Verificación de la Firma

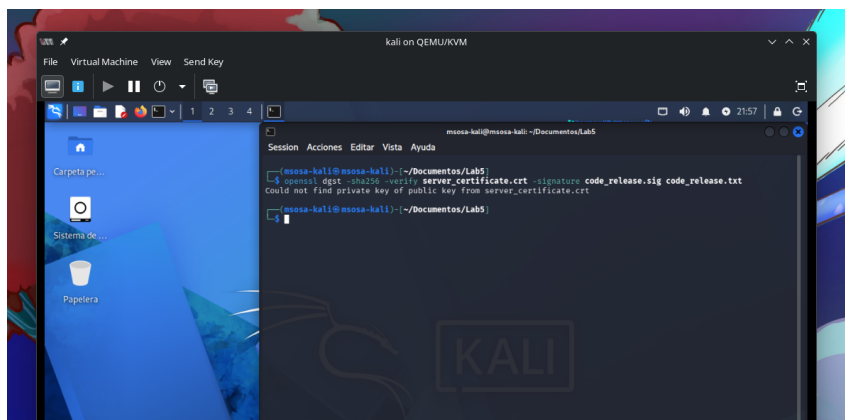


Figure 22: Prueba de Integridad

Conclusiones Esperadas de la Práctica

Al finalizar el laboratorio, el estudiante debe ser capaz de elaborar las siguientes conclusiones técnicas:

1. **Confirmación de la PKI:** Se verificó que la **Infraestructura de Clave Pública (PKI)** se construye exitosamente a partir de un **par de claves** y una **solicitud de firma (CSR)**, siendo el certificado digital el archivo que valida la **clave pública** del propietario.
2. **Diferenciación Criptográfica:** Se demostró que el cifrado **simétrico (AES)** es más adecuado para la encriptación de grandes volúmenes de datos por su velocidad, mientras que el cifrado **asimétrico (RSA)** es indispensable para el intercambio seguro de la clave simétrica y para funciones de autenticación.
3. **Garantía de Integridad y No Repudio:** Se comprobó que la **firma digital** no solo asegura la **integridad** de un documento (al detectar cualquier alteración mediante el fallo de la verificación del **hash**), sino que también garantiza la **autenticidad** y el **no repudio**, ya que solo la clave privada del firmante pudo haber generado dicha firma.

Referencias Bibliográficas

- OpenSSL. (2024). *OpenSSL Cryptography and SSL/TLS Toolkit*. <https://soporte.itlinux.cl/hc/es/articles/202497479-Comando-%C3%BAtiles-de-OpenSSL>.
- Stallings, W. (2020). *Cryptography and Network Security: Principles and Practice* (8th ed.). Pearson. (Referencia a los principios de PKI y firmas digitales).